



UNIVERSITÀ DEGLI STUDI DI FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

ChargeNet

Piattaforma per la gestione di stazioni di ricarica EV

Candidati:

Tommaso Sottili 7138541

Luca Moracci 7140812

Marco Lampronti 7147400

Corso:

Ingegneria del software

Docente:

Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Descrizione	3
1.2	Stack Tecnologico	3
1.3	Semplificazioni adottate	4
2	Analisi dei Requisiti e Casi d'uso	5
2.1	Use Case Diagram	5
2.2	Use Case Templates	6
2.3	Mockups	10
2.4	Page Navigation Diagram	10
2.5	Package Diagram	10
2.6	Class Diagram	11
2.6.1	Domain Model	11
2.6.2	Business Layer	12
2.6.3	DAO	12
2.6.4	Presentation Layer	13
2.7	Sequence Diagram	13
2.7.1	Ciclo di vita del GridMonitor	14
2.7.2	Approvazione di una colonna	15
2.7.3	Gestione dell'evento di temperatura troppo alta (Thermal Alert)	16
2.8	Database	17
2.8.1	Schema ER	18
2.8.2	Modello Relazionale	19
3	Implementazione	22
3.1	Domain Model	22
3.1.1	Users	23
3.1.2	Infrastructures	24
3.1.3	Sessions	25
3.1.4	Financials	26
3.1.5	Feedback	26
3.2	Business Layer	27
3.2.1	GridCluster	27
3.2.2	GridMonitor	28
3.2.3	LoadBalancerService	29
3.2.4	SessionService	32
3.2.5	StationService	34
3.2.6	Sicurezza e Validazione	34

3.2.7	Gestione Utente e Portafoglio	36
3.2.8	Gestione Feedback e Manutenzione	36
3.3	Controllers	38
3.4	Persistenza: il livello DAO e il mapping oggetti/tabelle	41
3.5	Idiomi e Design Pattern	43
3.5.1	Observer Pattern	44
3.5.2	Strategy Pattern	44
3.5.3	Factory Method	44
3.5.4	Singleton	45
3.5.5	DAO (Data Access Object)	47
3.5.6	Unit of Work	47
3.5.7	Service Locator	47
4	Testing	48
4.1	Strategia di Verifica	48
4.1.1	Tecnologie e Strumenti	48
4.2	Livello 1: Business Logic	48
4.2.1	Logica pura, senza mock — <code>ChargingStrategiesTest</code>	48
4.2.2	Singleton e reset dello stato : <code>GridClusterTest</code>	50
4.2.3	Service e gestione transazionale : <code>DriverServiceTest, LoadBalancerServiceTest</code>	50
4.2.4	Il cuore economico: <code>SessionServiceTest</code>	53
4.2.5	La concorrenza sulle prenotazioni: <code>StationServiceTest</code>	54
4.3	Livello 2: Persistence Layer (DAO)	54
5	Demo	56

Capitolo 1

Introduzione

1.1 Descrizione

Il progetto **ChargeNet** è un sistema software pensato per la gestione di una rete di stazioni di ricarica per veicoli elettrici (EV). L'obiettivo è coordinare l'intera sessione di ricarica: dalla ricerca di una colonnina libera, alla gestione del pagamento, fino alla tutela dell'infrastruttura elettrica da eventuali danni.

La piattaforma è pensata per tre figure, ciascuna con le proprie funzionalità:

- **Driver:** È l'utente finale. Utilizza il sistema per cercare le stazioni compatibili con la sua auto, far partire e monitorare la ricarica, e gestire i pagamenti tramite un portafoglio virtuale interno (wallet).
- **Station Operator:** È il proprietario delle colonnine. Registra nella piattaforma le nuove stazioni con i loro parametri tecnici (potenza, connettore, quartiere) e la tariffa che intende applicare, e ne segue l'andamento economico: guadagni maturati, energia venduta e numero di sessioni erogate. Non può decidere però da solo se una colonnina entra in rete, infatti ogni nuova registrazione resta in attesa dell'approvazione dell'Energy Manager.
- **Energy Manager:** È il supervisore della rete e ha un doppio ruolo. Da un lato autorizza, vale a dire approva o rifiuta le colonnine proposte dagli operatori e, approvandole, ne fissa la quota di tariffa trattenuta dalla piattaforma; interviene inoltre sulle colonnine segnalate per qualità scadente, sospendendole o archiviando la segnalazione. Dall'altro ha il compito di supervisionare. Ha infatti una dashboard che gli mostra in tempo reale carico e temperatura di ogni trasformatore. La protezione dell'infrastruttura in caso di surriscaldamento non è però una sua azione manuale: è il sistema stesso a reagire in autonomia interrompendo le ricariche sul ramo in sovraccarico (*Load Balancing*, descritto nel paragrafo 3.5.1).

1.2 Stack Tecnologico

- **Linguaggio:** Java, essendo un linguaggio fortemente orientato agli oggetti, ci ha permesso di modellare in modo chiaro le entità del dominio e le loro interazioni.
- **Interfaccia grafica:** JavaFX con FXML, che separa la descrizione del layout (i file `.fxml`) dalla logica di schermata (i **Controller**).

- **Persistenza:** database relazionale PostgreSQL. In sviluppo si è usato H2 in modalità `MODE=PostgreSQL`, così da accettare la stessa sintassi SQL e rendere veloce e semplice il passaggio a PostgreSQL.
- **Accesso ai dati:** JDBC con query scritte a mano, senza ORM, per controllo esplicito della traduzione oggetti/tabelle.
- **Sicurezza:** password protette con hash BCrypt.
- **Build e dipendenze:** Maven.
- **Test:** JUnit 5 con Mockito per l'isolamento tramite mock.

1.3 Semplificazioni adottate

Trattandosi di un progetto universitario, con tempo e risorse limitati, sono state adottate alcune semplificazioni, cercando comunque di restare vicini a un sistema realmente utilizzabile. Le scelte principali sono le seguenti:

- La geolocalizzazione non deriva da un indirizzo reale ma dal quartiere scelto (per le colonnine) o da una lettura GPS simulata (per il Driver); in produzione si userebbe un servizio di geocoding.
- La distanza tra due punti è calcolata con formula euclidea, più che sufficiente all'unico scopo di ordinare le colonnine per vicinanza.
- Non esiste un sistema di pagamento verso l'esterno: ricariche del portafoglio e guadagni sono movimenti interni.
- I coefficienti di calore dei trasformatori sono tarati per rendere il surriscaldamento osservabile in un tempo breve. Nella realtà un trasformatore si surriscalda per l'effetto cumulativo di molte ricariche contemporanee sullo stesso ramo; nella nostra applicazione, pensata per un solo utente alla volta, non è possibile simulare più sessioni in parallelo per raggiungere lo stesso effetto. Per questo motivo il calore generato da una singola sessione è stato aumentato, così da poter osservare l'intero meccanismo (accumulo, soglia, notifica, reazione) anche con una sola ricarica attiva, senza alterarne la logica.

Capitolo 2

Analisi dei Requisiti e Casi d'uso

Questo capitolo descrive il processo di progettazione del sistema seguendo un approccio che va dall'esterno verso l'interno, partendo da come gli utenti interagiscono con l'applicazione, definendo i requisiti e l'interfaccia grafica, per poi scendere nei dettagli tecnici dell'architettura software e della struttura del database.

2.1 Use Case Diagram

Il diagramma dei casi d'uso offre una visione d'insieme delle funzionalità del sistema, mostrando quali azioni possono compiere i tre attori principali (Driver, Station Operator ed Energy Manager) all'interno della piattaforma.

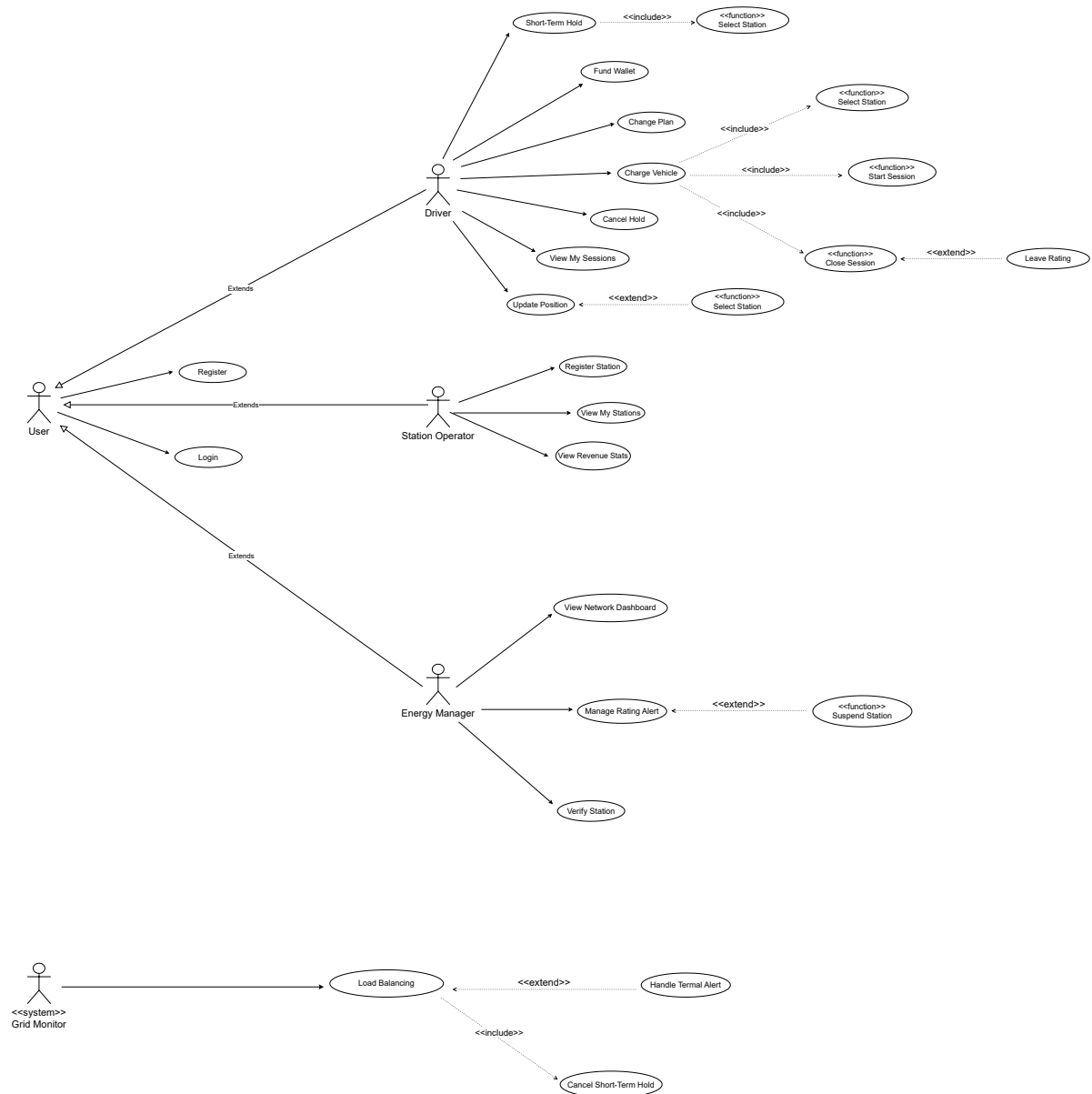


Figura 2.1: Diagramma dei casi d'uso

2.2 Use Case Templates

Per evitare ambiguità, i casi d'uso più importanti sono stati documentati con dei template strutturati, che descrivono passo dopo passo il flusso degli eventi, le precondizioni necessarie e i possibili scenari di errore.

Tabella 2.1: Caso d'Uso: Ricarica Veicolo

Campo	Contenuto
Nome	Ricarica Veicolo (Charge Vehicle)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente avvia una sessione di ricarica su una stazione selezionata, scegliendo una specifica strategia di ricarica (Fast o Eco). La sessione viene eseguita in tempo reale: il Grid Monitor la fa avanzare e l'interfaccia utente (Live Session UI) ne osserva lo stato tramite polling.
Pre-condizioni	Il Conducente ha effettuato l'accesso. Il saldo del Portafoglio (Wallet) del Conducente copre almeno il costo di 5 kWh presso la stazione selezionata, calcolato per il piano del Conducente (<code>station.priceFor(5.0, driver)</code>) e la strategia selezionata. La stazione selezionata è ACTIVE (oppure RESERVED dal Conducente) ed è compatibile con il connettore del veicolo.
Post-condizioni	La sessione è COMPLETED , il costo (detratto in modo incrementale per ogni tick durante la ricarica) è interamente addebitato sul Portafoglio, viene registrata una Transazione di tipo CHARGE e la stazione torna allo stato ACTIVE .
Attori	Conducente, Grid Monitor (Sistema)
Unit Test	<code>SessionServiceTest</code>

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Campo	Contenuto
Flusso Principale	1. Il Conducente seleziona una stazione dalla lista delle stazioni disponibili e compatibili più vicine (ACTIVE, o RESERVED da lui stesso). 2. Il Conducente apre la Live Session per quella stazione. 3. Il Conducente sceglie una strategia di ricarica: Fast Charge o Eco Charge. 4. Il Conducente inserisce la percentuale di batteria attuale del veicolo. 5. Il Conducente preme Start; il sistema valida il saldo del Portafoglio e la disponibilità della stazione, imposta la stazione su BUSY e apre la sessione. 6. Il Grid Monitor, ogni 5s, aggiorna batteria e kWh erogati, accumula il calore sul trasformatore di alimentazione e deduce i fondi in base alla strategia (lo sconto del piano si applica solo alla quota tariffaria della piattaforma). 7. La Live Session UI interroga (poll) lo stato della sessione e riflette in tempo reale batteria, kWh, costo attuale e Portafoglio. 8. Il Conducente preme Stop (oppure la sessione si auto-completa quando la batteria raggiunge il 100%). 9. Il sistema calcola i kWh totali erogati, il costo finale e registra la Transazione. 10. Il sistema reimposta lo stato della stazione su ACTIVE e mostra il popup di valutazione (rating).
Flussi Alternativi	1. Stazioni non disponibili nascoste: 1.1. Le stazioni BUSY, OVERLOADED, incompatibili o RESERVED da altri non sono mostrate in lista, impedendo al Conducente di selezionare una stazione non disponibile. 5. Saldo insufficiente: 5.1. Se il saldo è inferiore al costo di 5 kWh, il sistema nega l'avvio e mostra: "Credito insufficiente per avviare la sessione." (<code>InsufficientBalanceException</code>). 6-7. Allarme Termico (Observer attivato): 6.1. Se la temperatura del trasformatore supera i 90°C, il <code>LoadBalancerService</code> (Observer) imposta tutte le stazioni su quel trasformatore su OVERLOADED e chiude forzatamente le relative sessioni attive. 6.2. La sessione diventa INTERRUPTED_THERMAL e la UI mostra: "Arresto di emergenza: surriscaldamento del sistema". 6.3. Il sistema elabora un rimborso automatico per i kWh pagati ma non erogati.

Campo	Contenuto
Nome	Prenotazione a Breve Termine (Short-Term Hold)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente prenota temporaneamente una stazione di ricarica ACTIVE per 15 minuti per assicurarne la disponibilità al suo arrivo fisico.
Pre-condizioni	• Il Conducente ha effettuato l'accesso. • Il Conducente non ha già un'altra prenotazione attiva. • La stazione di destinazione è ACTIVE ed è compatibile con il connettore del Conducente.
Post-condizioni	Lo stato della stazione viene modificato in RESERVED , reserved_by_id viene impostato sul Conducente e expiration_timestamp viene impostato all'orario attuale più 15 minuti. La stazione prenotata rimane visibile al Conducente che ha effettuato la prenotazione (contrassegnata come RESERVED) e viene nascosta a tutti gli altri Conducenti.
Attori	Conducente
Unit Test	StationServiceTest
Flusso Principale	1. Il Conducente seleziona una stazione ACTIVE dalla lista delle stazioni disponibili. 2. Il Conducente preme Smart Book per confermare la prenotazione. 3. Il sistema verifica che la stazione sia ancora ACTIVE (controllo di concorrenza). 4. Il sistema imposta lo stato della stazione su RESERVED e reserved_by_id sul Conducente. 5. Il sistema imposta expiration_timestamp all'orario attuale più 15 minuti. 6. Il sistema mostra un messaggio di successo; la stazione appare ora come RESERVED nella lista del Conducente, offrendo solo l'opzione Charge Now.
Flussi Alternativi	3. Stazione non disponibile (Conflitto di concorrenza): 3.1. Se la stazione è stata prenotata o occupata da un altro utente nel frattempo, il sistema nega la prenotazione. 3.2. Il sistema mostra il messaggio: "Stazione non più disponibile". 3.3. Ritorno al passo 1. Nota sulla Scadenza: • Se trascorrono i 15 minuti senza che il Conducente avvii una sessione, il Grid Monitor ripristina automaticamente la stazione su ACTIVE (gestito dal caso d'uso Expire Short-Term Holds).

Tabella 2.2: Caso d'Uso: Prenotazione a Breve Termine

2.3 Mockups

In questa fase sono stati realizzati i mockup dell'interfaccia grafica, per verificare che le funzionalità descritte nei casi d'uso fossero accessibili in modo semplice e intuitivo.

2.4 Page Navigation Diagram

Il diagramma di navigazione illustra in che modo le diverse schermate dell'applicazione sono collegate tra loro, mostrando il percorso che l'utente compie muovendosi nel sistema in base alle scelte effettuate o ai pulsanti premuti.

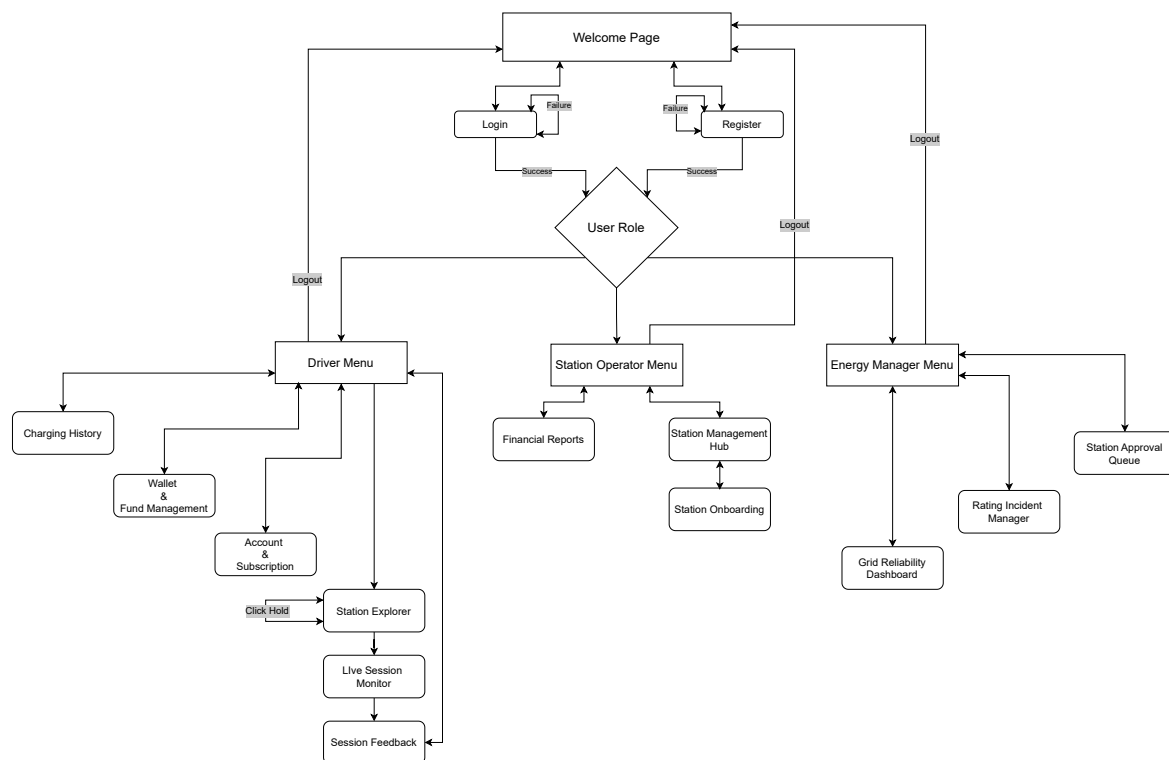


Figura 2.2: Diagramma di navigazione tra le pagine dell'applicazione.

2.5 Package Diagram

Passando all'architettura interna, il codice è organizzato in **quattro livelli sovrapposti** sotto il package base `it.unifi.ing.chargenet`, ciascuno con una responsabilità precisa:

- **presentation** — tutto ciò che l'utente vede: i Controller delle schermate JavaFX e il `NavigationManager`, che decide quale schermata mostrare. Non sa nulla di query o di database: chiede ai servizi di fare qualcosa e ne mostra il risultato.
- **business** — la logica applicativa: i `Service`, le strategie di ricarica e i componenti che fanno girare la simulazione (`GridMonitor`, `GridCluster`, `LoadBalancerService`). È qui che vivono le regole del *cosa deve succedere*.

- **dao** — la traduzione fra oggetti Java e righe di tabella. È l'unico livello che sa com'è fatto il database.
- **domain** — le entità: `ChargingStation`, `ChargingSession`, `Driver`, `PowerTransformer` e le altre. Non sono contenitori di dati passivi, ma oggetti *ricchi*, che sanno fare ciò che riguarda la loro natura: è la `ChargingStation` stessa a sapere come si calcola il prezzo di una ricarica, ed è la `ChargingSession` stessa a sapere a quali condizioni può essere aperta.

Il dominio non dipende dagli altri livelli

Nella progettazione dell'architettura è stato rispettato il principio per cui nessuna classe del livello **domain** possa dipendere da un altro livello. Le entità non sanno che esistono servizi, controller o database, e nel diagramma questo si può leggere subito: verso il package **domain** entrano frecce da tutti gli altri livelli, ma da esso non ne esce nessuna. Non è solo un'intenzione, ma una proprietà verificabile: nell'intero package **domain** non compare un solo `import` di **business**, **dao** o **presentation**.

Il rispetto di tale principio ha avuto conseguenze concrete. Per esempio il calcolo del prezzo base di una ricarica è stato messo sulla `ChargingStation` (dominio puro) e non sull'interfaccia delle strategie, che si trova nel **business**, proprio perché così sia il dominio che il **business** possono usarlo senza che il dominio debba rivolgersi esternamente. La scelta opposta lo avrebbe reso dipendente da uno strato più esterno e più volatile.

Il beneficio si è visto in modo tangibile nella migrazione da H2 a PostgreSQL, dove entità, servizi e controller sono rimasti intatti perché nessuno di loro sapeva quale database ci fosse sotto: le uniche cose toccate sono state la configurazione della connessione e il bootstrap del database.

2.6 Class Diagram

Il sistema è stato progettato con un'architettura a strati, per mantenere alta coesione e basso accoppiamento (*high cohesion and low coupling*). Nei paragrafi seguenti sono approfondite le classi principali di ciascun livello.

2.6.1 Domain Model

Questo diagramma rappresenta il Domain Model, cioè le entità principali su cui si basa il sistema: utenti, colonnine, trasformatori, sessioni di ricarica e gli oggetti ad esse collegati. Una descrizione più dettagliata sul Domain Model è presente nel paragrafo 3.1

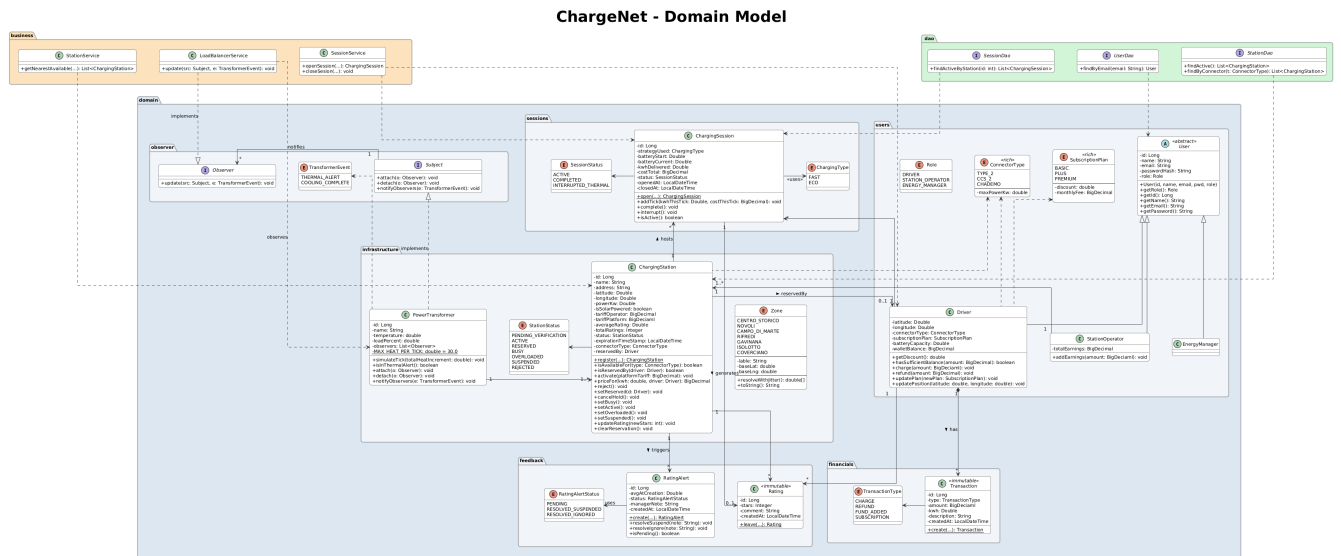


Figura 2.3: Domain Model, diagramma di classe

2.6.2 Business Layer

Il Business Layer è il livello che contiene la logica applicativa del sistema, dove si trovano i servizi che coordinano le entità del Domain Model, le strategie di ricarica e i componenti che gestiscono la simulazione della rete. Il livello è approfondito nel paragrafo 3.2.

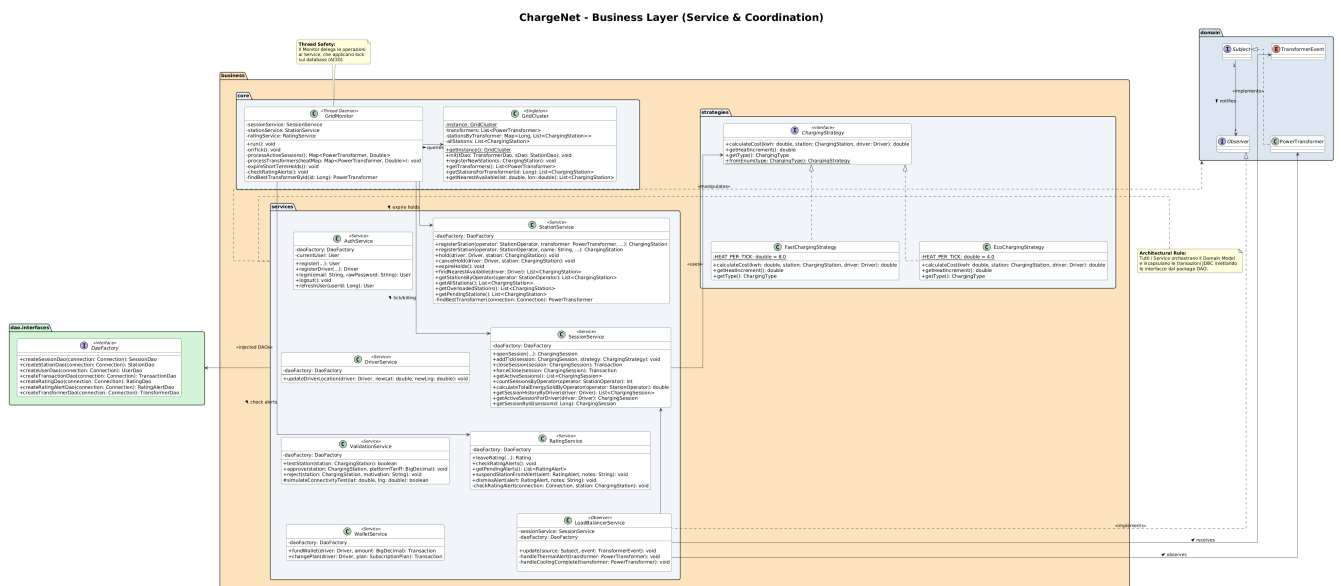


Figura 2.4: Business Layer, diagramma di classe

2.6.3 DAO

Il livello DAO si occupa della comunicazione con il database, traducendo le entità del Domain Model in dati salvabili e recuperabili e garantendone la persistenza.

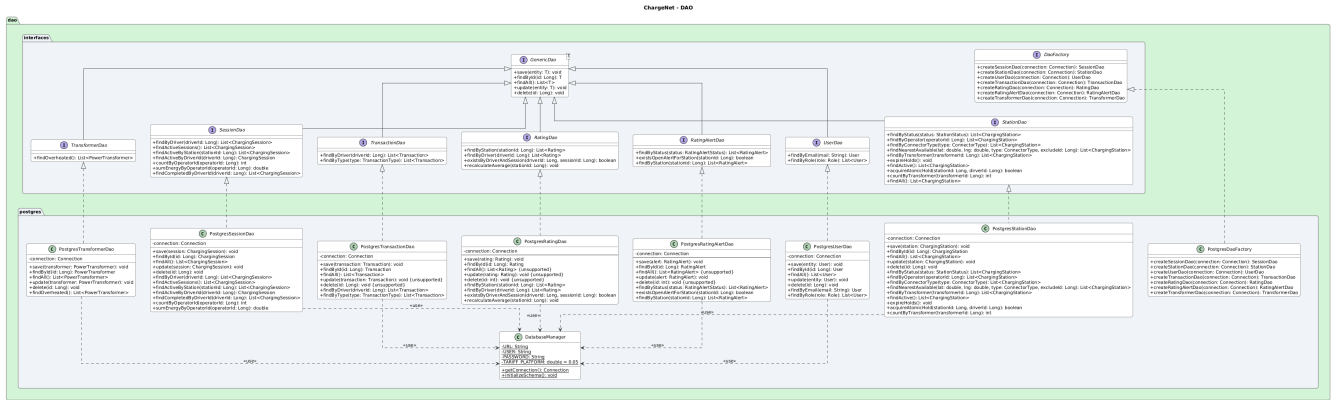


Figura 2.5: DAO, diagramma di classe

2.6.4 Presentation Layer

Il seguente diagramma rappresenta il presentation layer, cioè l'interfaccia grafica dell'applicazione realizzata in JavaFX. Comprende i controller delle varie schermate, che si occupano di mostrare i dati e raccogliere le azioni dell'utente, appoggiandosi ai servizi del livello di business.

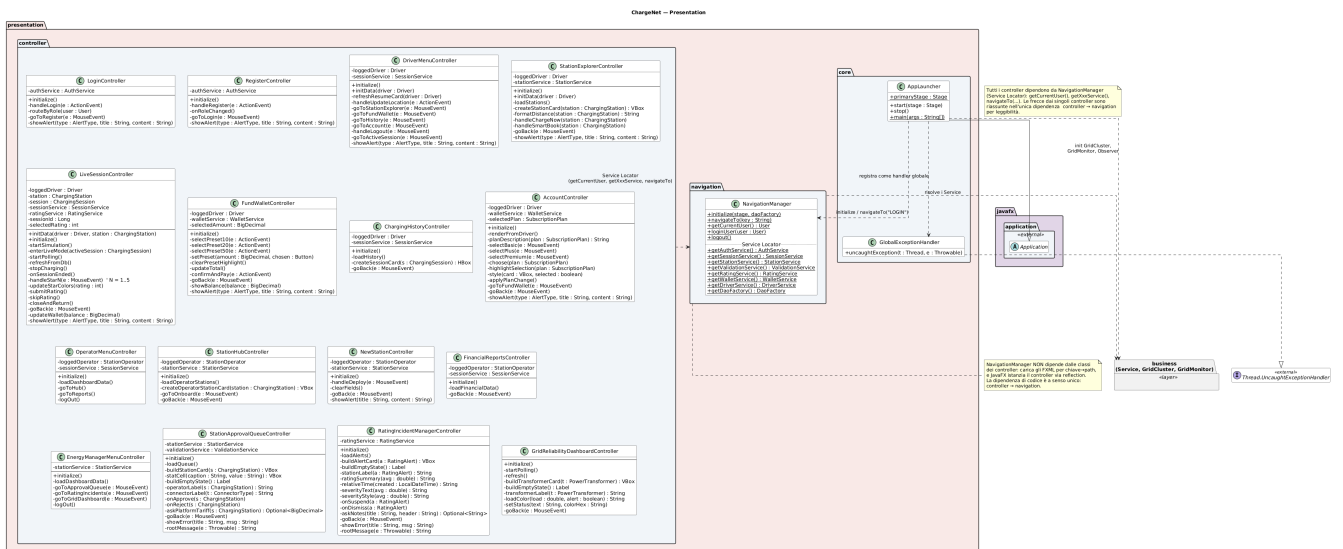


Figura 2.6: Presentation Layer, diagramma di classe

2.7 Sequence Diagram

I diagrammi di sequenza che seguono descrivono alcuni dei flussi più significativi del sistema, mostrando come le varie classi interagiscono tra loro nel tempo per portare a termine un'operazione.

2.7.1 Ciclo di vita del GridMonitor

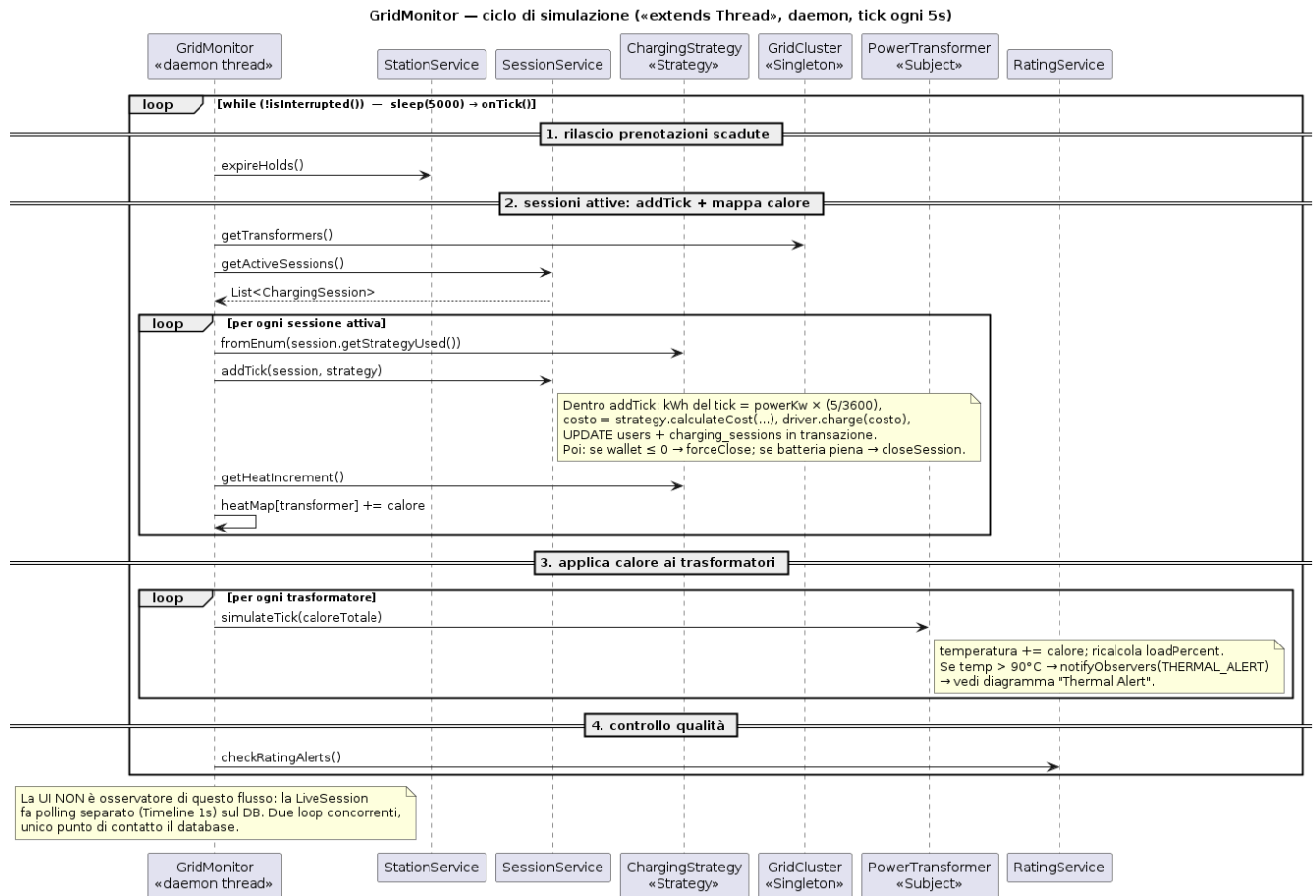


Figura 2.7: Diagramma di sequenza del ciclo di vita del GridMonitor.

Il **GridMonitor** (Fig. 2.7) è il thread daemon che scandisce l'intera simulazione della rete elettrica con un tick fisso di 5 secondi, ed è l'unico componente attivo di propria iniziativa nel sistema: tutti gli altri servizi vengono invocati su richiesta. Ad ogni ciclo il monitor libera prima le prenotazioni scadute tramite **StationService.expireHolds()**, poi recupera da **SessionService** l'elenco delle sessioni attive e, per ciascuna, seleziona la **ChargingStrategy** corrispondente per calcolare kWh erogati e costo del tick, aggiornando in un'unica transazione sia il wallet del Driver sia lo stato della sessione. Lo stesso tick genera un incremento di calore che viene accumulato per trasformatore e applicato a fine ciclo a ciascun **PowerTransformer**: se la temperatura supera i 90°C viene notificato l'evento **THERMAL_ALERT** agli observer registrati, dando origine al flusso descritto in Fig. 2.9. Infine il monitor invoca il controllo di qualità del **RatingService**. La UI non partecipa a questo ciclo come observer: lo stato delle sessioni viene mostrato tramite un polling separato sul database, in questo i due flussi restano disaccoppiati e l'unico punto di sincronizzazione è la persistenza.

2.7.2 Approvazione di una colonnina

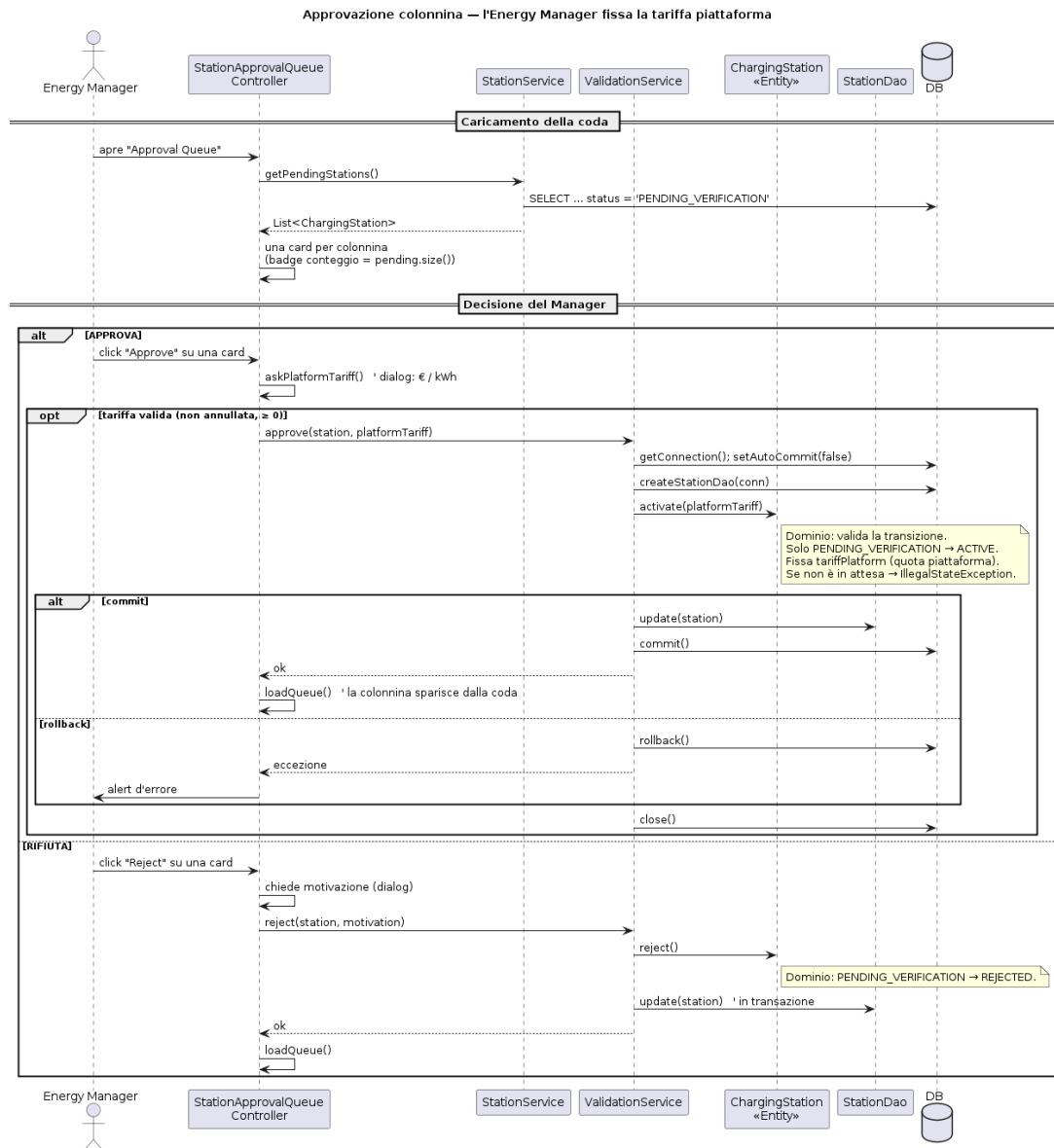


Figura 2.8: Diagramma di sequenza dell'approvazione di una colonnina da parte dell'Energy Manager.

Il flusso di approvazione (Fig. 2.8) regola l'ingresso in esercizio di una nuova colonnina proposta da uno Station Operator. L'Energy Manager apre la coda delle richieste in stato `PENDING_VERIFICATION`, occupata da `StationService.getPendingStations()`, e per ciascuna richiesta può approvare o rifiutare. In caso di approvazione il controller richiede la tariffa di piattaforma da applicare in €/kWh e delega la transizione alla `ValidationService`, che apre una connessione con `setAutoCommit(false)` prima di invocare `activate()` sul dominio: è la `ChargingStation` stessa a validare che la transizione sia legittima solo a partire da `PENDING_VERIFICATION`, sollevando un'eccezione altrimenti, mentre il servizio si occupa unicamente di gestire la persistenza e il commit o il rollback della transazione in caso di errore. La stessa separazione si ritrova nel percorso di rifiuto, dove il dominio applica la transizione a `REJECTED` dopo che il Manager ha fornito una

motivazione. In entrambi i casi la responsabilità di stabilire se un cambio di stato è ammesso resta interamente nel Domain Model, mentre il Business Layer si limita a decidere quando invocarla e a garantirne l'atomicità della persistenza.

2.7.3 Gestione dell'evento di temperatura troppo alta (Thermal Alert)

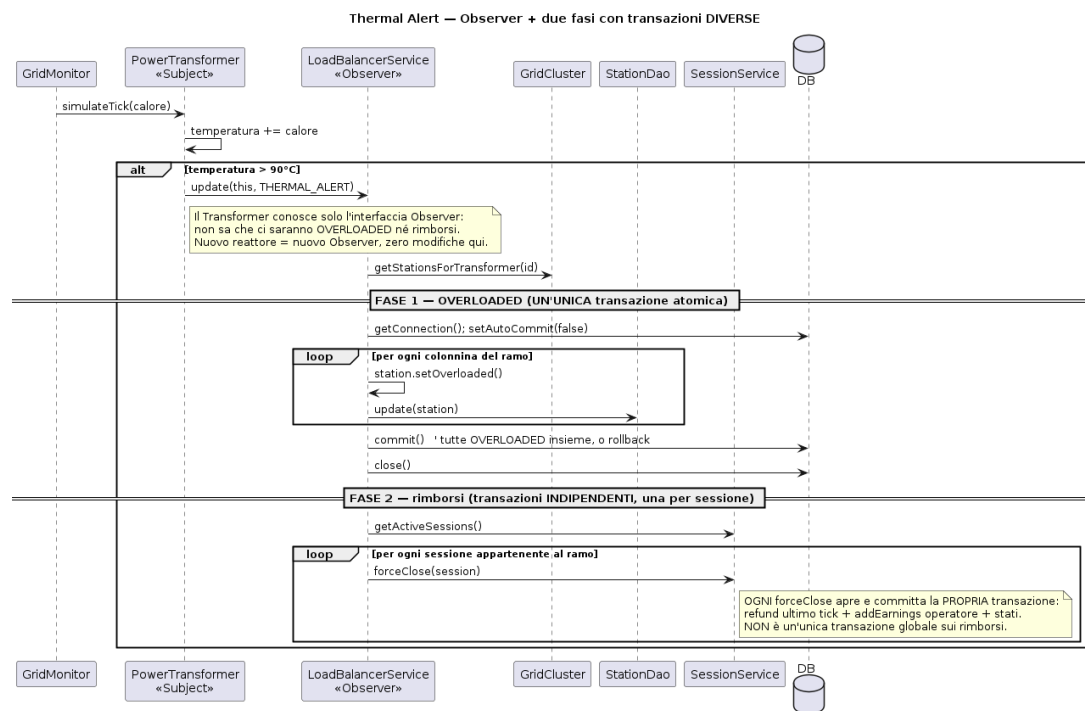


Figura 2.9: Diagramma di sequenza della gestione di una temperatura troppo alta (Thermal Alert).

Il diagramma di Thermal Alert (Fig. 2.9) mostra cosa succede quando un trasformatore si surriscalda. Durante il tick del `GridMonitor`, il `PowerTransformer` riceve il nuovo carico di calore e aggiorna la propria temperatura interna. Se questa supera i 90°C, il trasformatore avvisa il `LoadBalancerService`, che è l'unico componente incaricato di reagire all'emergenza: il trasformatore stesso non sa cosa succederà dopo, si limita a segnalare il problema.

A quel punto il `LoadBalancerService` interviene in due passaggi separati. Nel primo, blocca subito tutte le colonnine collegate a quel trasformatore mettendole in stato `OVERLOADED`, per evitare che qualcuno continui a ricaricare su un ramo elettrico in sovraccarico: questo blocco viene fatto tutto insieme, colonnina per colonnina, come un'unica operazione che riesce o fallisce nel suo complesso. Nel secondo passaggio, il sistema chiude una per una tutte le sessioni di ricarica che erano attive su quelle colonnine, rimborsando ogni Driver coinvolto tramite `SessionService.forceClose()`. Qui però ogni chiusura viene trattata singolarmente: se il rimborso di una sessione dovesse fallire, le altre sessioni verrebbero comunque chiuse e rimborsate correttamente, e soprattutto le colonnine resterebbero comunque bloccate in sicurezza, indipendentemente da come va il rimborso.

2.8 Database

La gestione dei dati di ChargeNet è affidata a un database relazionale PostgreSQL. In fase di sviluppo si è però lavorato con **H2**, un database che può girare interamente in memoria, che ci è tornato comodo in quanto non richiede installazione e si avvia e si spegne con l'applicazione. Questo ha permesso di scrivere query e provare l'applicazione da subito, rimandando l'installazione e la configurazione del database reale.

Per poter fare poi il passaggio abbiamo dovuto usare H2 fin dall'inizio in modalità `MODE=PostgreSQL`, e scrivere lo schema con tipi e sintassi standard di Postgres (`BIGSERIAL` per le chiavi auto-incrementanti, `DECIMAL`, `TIMESTAMP`, `BOOLEAN`). In questo modo non abbiamo dovuto riscrivere nessuna query dei DAO e nessun Service o Controller si è accorto del cambiamento. E' una dimostrazione concreta dell'indipendenza del dominio descritta nel paragrafo 2.5.

Un'altra operazione necessaria per la migrazione è stata rendere il bootstrap **idempotente**: le istruzioni `DROP TABLE` sono state eliminate in favore di `CREATE TABLE IF NOT EXISTS`, e soprattutto il *seed* dei dati iniziali viene eseguito solo se il database risulta vuoto, con un controllo esplicito che conta le righe della tabella `users`.

Listing 2.1: Bootstrap idempotente: lo schema si crea se manca, il seed solo su DB vuoto

```
// Ri-seedare ad ogni avvio duplicherebbe tutto (e violerebbe
    UNIQUE su email)
if (isDatabaseEmpty(conn)) {
    seedData(conn);
} else {
    System.out.println("[DatabaseManager] DB gia' popolato, seed
        saltato.");
}
```

Un secondo avvio ritrova quindi gli stessi dati invece di duplicarli o azzerarli, che è precisamente la prova che la persistenza funziona.

2.8.1 Schema ER

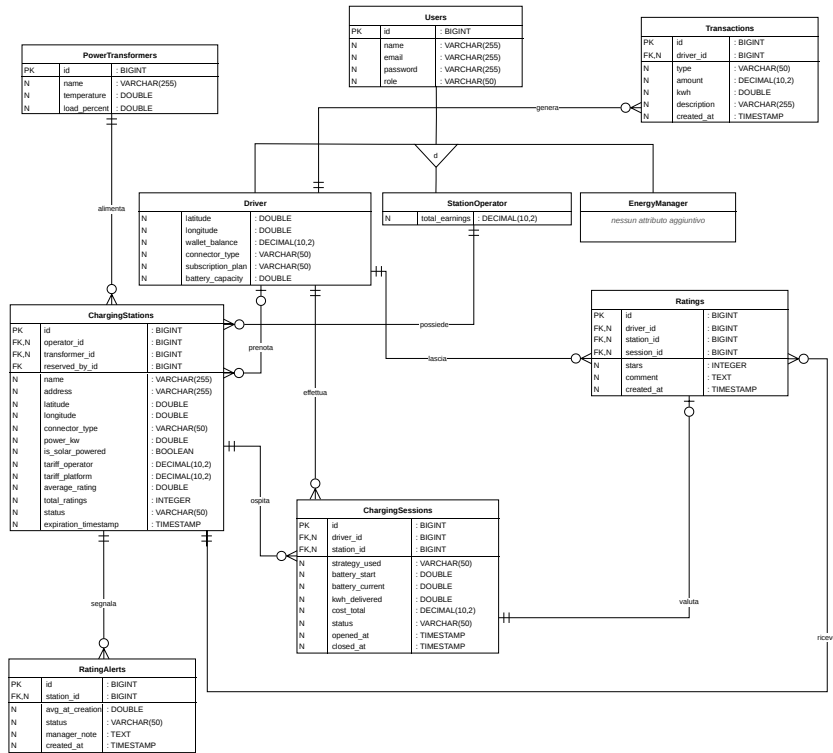


Figura 2.10: Schema Entità-Relazione del database ChargeNet.

Lo schema (Fig. 2.10) mostra la stessa gerarchia già introdotta nel Domain Model: **Users** è un'entità generica che rappresenta ciò che ogni utente ha in comune (identità, credenziali, ruolo), e viene specializzata in tre sottotipi, **Driver**, **StationOperator** ed **EnergyManager**, ciascuno con i propri attributi specifici. Un utente appartiene esattamente a uno dei tre sottotipi, mai a più di uno contemporaneamente, perché la colonna **role** può avere un solo valore.

Le relazioni principali partono dallo specifico sottotipo e non genericamente da **Users**:

- Un **PowerTransformer** alimenta più **ChargingStation**.
- Uno **StationOperator** possiede più **ChargingStation**.
- Un **Driver** può prenotare temporaneamente una colonnina, avviare più **ChargingSession**, generare più **Transaction** e lasciare più **Rating**.
- Una **ChargingStation** ospita più **ChargingSession** e può ricevere più **Rating** o generare più **RatingAlert**.
- Ogni sessione può generare al più una **Rating**.
- **EnergyManager** non compare in nessuna relazione dello schema: il suo ruolo (approvare colonnine, gestire gli alert di qualità) non richiede di possedere o generare entità proprie, ma di agire su entità già esistenti.

2.8.2 Modello Relazionale

La traduzione delle entità semplici segue le regole standard: ogni entità diventa una tabella, ogni relazione (1,n) diventa una chiave esterna sul lato "molti". La generalizzazione è stata invece tradotta con un'unica tabella che raccoglie tutte le colonne di tutti i sottotipi, dove le colonne che non riguardano una riga restano NULL e una colonna discriminatore indica il sottotipo reale.

```
Users(id, name, email, password, role, latitude, longitude, wallet_balance,  
      connector_type, subscription_plan, battery_capacity, total_earnings)
```

```
PowerTransformers(id, name, temperature, load_percent)
```

```
ChargingStations(id, operator_id, transformer_id, name, address,  
                 latitude, longitude, connector_type, power_kw, is_solar_powered,  
                 tariff_operator, tariff_platform, average_rating, total_ratings,  
                 status, reserved_by_id*, expiration_timestamp)
```

```
ChargingSessions(id, driver_id, station_id, strategy_used,  
                 battery_start, battery_current, kwh_delivered, cost_total,  
                 status,  
                 opened_at, closed_at*)
```

```
Transactions(id, driver_id, type, amount, kwh, description, created_at)
```

```
Ratings(id, driver_id, station_id, session_id, stars,  
        comment, created_at)    UNIQUE(driver_id, session_id)
```

```
RatingAlerts(id, station_id, avg_at_creation, status, manager_note*,  
             created_at)
```

dove:

`operator_id, reserved_by_id` si riferisce a `Users(id)`
`transformer_id` si riferisce a `PowerTransformers(id)`
`driver_id` si riferisce a `Users(id)`
`station_id` si riferisce a `ChargingStations(id)`
`session_id` si riferisce a `ChargingSessions(id)`

Legenda della notazione

attributo — Chiave Primaria (PK) della tabella.

attributo — Chiave Esterna (FK) verso un'altra tabella.

* — Attributo opzionale (NULL ammesso); tutti gli altri attributi sono obbligatori (NOT NULL).

UNIQUE(...) — Vincolo di unicità su una combinazione di più colonne.

La colonna `role` in `Users` dice se la riga rappresenta un `Driver`, uno `StationOperator` o un `EnergyManager`. Colonne come `wallet_balance` (`Driver`) o `total_earnings` (`StationOperator`) esistono per tutte le righe della tabella, ma restano `NULL` per i sottotipi a cui non appartengono: per esempio non ha senso che `EnergyManager` abbia un valore diverso da `null` nell'attributo `subscription_plan`. È lo stesso meccanismo descritto nel paragrafo 3.4 a proposito del metodo `extractUserFromResultSet()`, che legge la colonna `role` e ricostruisce l'oggetto Java del sottotipo corretto.

Le due FK verso `Users` in `ChargingStations` (`operator_id` e `reserved_by_id`) modellano invece due relazioni indipendenti — chi possiede la colonnina e chi la sta eventualmente prenotando — ed è per questo che solo la seconda può essere nulla.

Il vincolo `UNIQUE (driver_id, session_id)` sulla tabella `Ratings` fa sì che anche a livello di database valga la regola che un `Driver` può lasciare una sola valutazione per ciascuna sessione.

Infine, diversi attributi numerici sono protetti da vincoli `CHECK` direttamente nello schema (es. `stars` tra 1 e 5, `wallet_balance` sempre ≥ 0): questi vincoli sono una seconda difesa rispetto alla validazione già presente nel Domain Model, indipendente da eventuali bug nel codice Java.

È presente un limite, conseguenza della generalizzazione utilizzata: la foreign key `operator_id` garantisce solo che l'id referenziato esista in `Users`, non che quella riga appartenga effettivamente a uno `StationOperator`; lo stesso vale per `driver_id` rispetto ai `Driver`. Questo vincolo di coerenza fra ruolo e utilizzo non è esprimibile a livello di schema relazionale standard.

Nell'applicativo il problema è comunque risolto dal sistema di tipi di Java, infatti `StationService.registerStation()` accetta come parametro uno `StationOperator`, non un generico `User`, e passargli un `Driver` non compilerebbe nemmeno. Quanto detto vale per il *codice applicativo* ma non per il database. Una `INSERT` scritta a mano o uno script di popolamento iniziale, possono benissimo scrivere in `operator_id` l'id di un `Driver`, e il database la accetterebbe subito, perché effettivamente quell'id in `Users` esiste davvero.

Non è solo un'ipotesi teorica, ma è proprio in questo modo che è passato un bug reale del seed, infatti alcune colonnine erano state attribuite per errore al driver invece che all'operatore, alterandoo poi le dashboard dei guadagni. La conclusione corretta è quindi che la coerenza fra ruolo e riferimento è garantita *dal codice*, non *dallo schema*.

E' efficace finché tutte le scritture passano dai Service, ma cade non appena qualcosa scrive nel database aggirandoli. Un'alternativa che avrebbe risolto il problema sarebbe stata tradurre la generalizzazione in tabelle separate per sottotipo, che però avrebbe comportato join aggiuntive su ogni login, motivo per cui si è preferita la semplicità della tabella unica, accettando consapevolmente questo limite.

Capitolo 3

Implementazione

Questo capitolo descrive le scelte implementative e l'architettura interna di ChargeNet. Per mantenere la trattazione chiara, non verrà mostrato l'intero codice sorgente, ma ci si concentrerà sull'organizzazione dei componenti principali e sui frammenti di codice che illustrano le logiche più complesse e i pattern utilizzati.

Architettura del Sistema: I livelli principali sono tre: Domain, Business e Controller. L'interazione con il database PostgreSQL è gestita tramite il pattern Data Access Object (DAO), che nasconde la complessità delle query SQL al resto dell'applicazione.

3.1 Domain Model

Per avere maggiore ordine, il Domain Model è stato suddiviso in cinque pacchetti principali, ognuno con una responsabilità specifica, raggruppando il codice in base alla finalità e al contesto di utilizzo.

I cinque pacchetti principali sono:

- **Utenti:** Definisce gli attori del sistema. Ci sono tre profili principali (*Driver*, *StationOperator* ed *EnergyManager*), che estendono tutti una classe madre comune per semplificare le operazioni di login e registrazione.
- **Infrastrutture:** Rappresenta le parti fisiche della rete. Contiene le colonnine (*ChargingStation*) e i trasformatori (*PowerTransformer*), e serve a gestire parametri come la potenza massima erogabile o lo stato di occupazione.
- **Sessioni:** È il collegamento pratico tra l'utente e l'infrastruttura. La classe centrale è *ChargingSession*, che si occupa di far partire la ricarica e di calcolare quanti kWh vengono consumati col passare del tempo.
- **Finanze:** Si occupa dei pagamenti e del saldo degli utenti. Gestisce le ricevute (*Transaction*) create dopo una ricarica, un rimborso o il pagamento di un abbonamento, facendo in modo che lo storico contabile non possa essere modificato per sbaglio.
- **Feedback:** Gestisce le recensioni. Raccoglie i voti (*Rating*) lasciati dai guidatori e fa scattare in automatico un avviso di manutenzione (*RatingAlert*) se la media di una stazione si abbassa troppo.

Nei prossimi paragrafi vedremo più nel dettaglio alcune delle soluzioni implementative che abbiamo adottato in questi cinque moduli.

3.1.1 Users

Questo modulo definisce le identità degli attori tramite un'architettura basata sull'ereditarietà. Dalla classe astratta `User` derivano i tre profili specifici (`Driver`, `StationOperator` ed `EnergyManager`), ognuno con i propri attributi.

Per rispettare il principio dell'incapsulamento, la logica di business è integrata direttamente nelle entità. Un esempio pratico è la gestione economica del `Driver`: quando c'è da scalare del credito, è l'oggetto stesso a fare i controlli sul proprio saldo.

Listing 3.1: Controllo del saldo nel metodo `charge()`

```
public void charge(BigDecimal amount) {
    if (!hasSufficientBalance(amount)) {
        throw new InsufficientBalanceException("Saldo insufficiente");
    }
    this.walletBalance = this.walletBalance.subtract(amount);
}
```

Questo approccio torna utile anche per operazioni più delicate. Il cambio di piano, ad esempio, non addebita il canone intero del nuovo piano, ma solo la differenza rispetto a quello attuale, e blocca i downgrade riusando comunque `charge()` per il controllo del saldo:

Listing 3.2: Aggiornamento del piano tariffario

```
public BigDecimal updatePlan(SubscriptionPlan newPlan) {
    BigDecimal currentFee = (this.subscriptionPlan != null)
        ? this.subscriptionPlan.getMonthlyFee() : BigDecimal.ZERO;
    BigDecimal delta = newPlan.getMonthlyFee().subtract(currentFee)
        .setScale(2, RoundingMode.HALF_UP);

    if (delta.compareTo(BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException(
            "Non è possibile passare a un piano inferiore durante il periodo attivo.");
    }
    if (delta.compareTo(BigDecimal.ZERO) > 0) {
        charge(delta);
    }
    this.subscriptionPlan = newPlan;
    return delta;
}
```

Per il collegamento con il database, invece di passare per i costruttori standard (che farebbero scattare validazioni inutili in fase di caricamento), abbiamo dotato ogni entità

di un metodo `reconstitute()`, pensato esclusivamente per ripristinare in memoria i dati letti da PostgreSQL.

3.1.2 Infrastructures

L'obiettivo di questo pacchetto è tradurre in codice i limiti fisici del mondo reale. Per la classe `ChargingStation` era necessario impedire la creazione di oggetti con parametri incoerenti o pericolosi.

Abbiamo quindi nascosto i costruttori, obbligando il resto del sistema a passare per il Factory Method `register()`. In questa fase la stazione esegue quattro controlli in sequenza: coordinate geografiche valide, potenza compresa tra 7 e 350 kW, potenza compatibile col limite fisico del connettore, e tariffa dell'operatore positiva. Basta che uno fallisca per bloccare subito la creazione.

Listing 3.3: Factory Method per la registrazione della stazione

```
public static ChargingStation register(StationOperator operator,
    PowerTransformer transformer,
                                   String name, String address
                                   , Double lat, Double lng
                                   ,
                                   ConnectorType connectorType
                                   , Double powerKw,
                                   Boolean isSolar, BigDecimal
                                   tariff) {
    if (lat < -90.0 || lat > 90.0) {
        throw new InvalidStationParametersException("Latitudine non
            valida");
    }
    if (lng < -180.0 || lng > 180.0) {
        throw new InvalidStationParametersException("Longitudine
            non valida");
    }
    if (powerKw < 7.0 || powerKw > 350.0) {
        throw new InvalidStationParametersException("Potenza non
            valida. Deve essere compresa tra 7 kW e 350 kW.");
    }
    if (powerKw > connectorType.getMaxPowerKw()) {
        throw new InvalidStationParametersException("Errore di
            sicurezza: La potenza richiesta (" + powerKw +
            " kW) supera il limite fisico supportato dal
            connettore " + connectorType.name() +
            " (" + connectorType.getMaxPowerKw() + " kW).");
    }
    if (tariff.compareTo(BigDecimal.ZERO) <= 0) {
        throw new InvalidStationParametersException("La tariffa
            dell'operatore deve essere strettamente positiva.");
    }
    return new ChargingStation(operator, transformer, name, address
        , lat, lng, connectorType, powerKw, isSolar, tariff);
}
```

L'altra parte fondamentale del pacchetto è il **PowerTransformer**, che fa da base per il sistema di Load Balancing. Piuttosto che avere un servizio esterno che controlla continuamente i trasformatori, abbiamo reso il trasformatore intelligente applicando il pattern Observer. A ogni ciclo di simulazione, ricalcola la sua temperatura interna; se supera la soglia critica dei 90 gradi, genera autonomamente un **THERMAL_ALERT** per avvisare la rete, senza bisogno di sapere chi o come gestirà effettivamente l'emergenza (si veda il paragrafo 3.5.1).

3.1.3 Sessions

Il pacchetto racchiude la logica legata all'erogazione dell'energia. La classe centrale, **ChargingSession**, non si limita a registrare passivamente i dati (come kWh erogati e costi parziali), ma regola attivamente l'intero ciclo di vita dell'operazione, dalla creazione fino al completamento naturale o all'interruzione per emergenza termica.

Per garantire che una ricarica non inizi in condizioni non valide, l'istanziamento è affidato al metodo `factory open()`. Questo metodo esegue una serie di controlli incrociati di dominio: verifica la compatibilità fisica del connettore, controlla che la stazione non sia occupata e delega a `ChargingStation.priceFor()` il calcolo del saldo minimo richiesto, lo stesso metodo che userà poi la strategia di ricarica scelta, così il controllo d'ingresso e l'addebito reale non possono mai calcolare cifre diverse:

Listing 3.4: Apertura di una sessione di ricarica

```
public static ChargingSession open(Driver driver, ChargingStation
    station, ChargingType strategy, Double batteryStart) {

    boolean isAvailable = station.isAvailableFor(driver.
        getConnectorType()) || station.isReservedBy(driver);
    if (!isAvailable) {
        throw new StationNotAvailableException("La colonnina " +
            station.getName() + " non e' disponibile o non e'
            compatibile col tuo veicolo.");
    }

    BigDecimal minimumRequired = station.priceFor(5.0, driver);

    if (!driver.hasSufficientBalance(minimumRequired)) {
        throw new InsufficientBalanceException("Credito
            insufficiente per avviare la sessione. E' richiesto un
            saldo di almeno " +
                minimumRequired + "\\texteuro.");
    }

    return new ChargingSession(driver, station, strategy,
        batteryStart);
}
```

Inoltre, la classe incapsula la logica di avanzamento tramite il metodo `addTick()`, che aggiorna in tempo reale i costi e l'energia immessa, facendo transitare automaticamente lo stato della sessione a completato qualora la batteria raggiunga il 100% della sua capacità.

3.1.4 Financials

Questo modulo gestisce lo storico dei movimenti economici degli utenti, categorizzandoli tramite l'enumerazione `TransactionType` (ricariche, rimborsi, abbonamenti e depositi).

La scelta architetturale più rilevante per l'entità `Transaction` è stata quella di renderla immutabile. Trattandosi di dati contabili, uno storico non doveva poter essere alterato accidentalmente a runtime: per questo la classe è sprovvista di metodi *setter* e i suoi campi sono valorizzabili unicamente al momento della creazione.

Per blindare ulteriormente l'integrità del registro, la generazione di un nuovo movimento non avviene tramite un normale costruttore, ma è protetta da un metodo *factory* che scarta a priori qualsiasi operazione anomala, come ad esempio i pagamenti a importo zero o privi di causale:

Listing 3.5: Creazione di una transazione contabile

```
public static Transaction create(Driver driver, TransactionType
    type, BigDecimal amount,
                                Double kwh, String description) {

    if (amount == null || amount.compareTo(BigDecimal.ZERO) == 0) {
        throw new IllegalArgumentException("Errore: l'importo non
            puo' essere zero.");
    }
    if (description == null || description.trim().isEmpty()) {
        throw new IllegalArgumentException("Errore: la transazione
            deve avere una descrizione.");
    }
    return new Transaction(driver, type, amount, kwh, description);
}
```

Questa scelta semplifica la logica del livello Business, che può contare su uno storico finanziario coerente.

3.1.5 Feedback

A chiudere il Domain Model troviamo la parte dedicata alla qualità del servizio e al supporto. Qui abbiamo deciso di separare l'azione dell'utente (la recensione) dalla conseguente gestione amministrativa (l'allarme per la manutenzione).

Nella classe `Rating`, la creazione di un nuovo feedback è protetta da controlli rigorosi. Abbiamo implementato il metodo *factory* `leave()`, che non si limita a istanziare la recensione, ma lega quest'ultima alla sessione di ricarica effettiva e impedisce l'inserimento di voti fuori dal range previsto:

Listing 3.6: Creazione di un feedback utente

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session, Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un valore compreso tra 1 e 5 stelle.
            ");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Una parte interessante di questo modulo si trova nella classe **RatingAlert**. Quando la media di una stazione cala drasticamente, il sistema genera un avviso. Invece di affidare la chiusura di questo ticket a un servizio esterno, abbiamo inserito la logica di transizione di stato direttamente all'interno dell'entità. I metodi per sospendere la colonnina o ignorare l'allarme verificano prima che la segnalazione sia effettivamente ancora in sospeso, evitando che un operatore possa inavvertitamente alterare pratiche già chiuse. La decisione di generare l'alert, invece, non spetta al dominio ma al **RatingService** (vedi paragrafo 3.2.8), che dopo ogni nuova recensione ricalcola la media e verifica la soglia.

Listing 3.7: Chiusura di un alert di manutenzione

```
public void resolveSuspend(String note) {
    if (isPending()) {
        this.status = RatingAlertStatus.RESOLVED_SUSPENDED;
        this.managerNote = note;
    }
}
```

3.2 Business Layer

Il Business Layer rappresenta il livello in cui i dati del Domain Model vengono elaborati e trasformati in funzionalità concrete. L'architettura di questo strato è stata progettata seguendo il principio di singola responsabilità (SRP), suddividendo il codice in servizi specializzati (*Services*) e componenti di simulazione (*Core*).

L'implementazione delle diverse logiche di erogazione (*ChargingStrategy*) verrà approfondita nella successiva sezione dedicata ai Design Pattern.

Di seguito vengono analizzate nel dettaglio le classi centrali che governano e dirigono il flusso dell'applicazione.

3.2.1 GridCluster

Il primo problema tecnico che abbiamo affrontato progettando la simulazione è stato il carico sul database. Poiché il sistema deve ricalcolare i consumi e le temperature della rete ogni pochi secondi, eseguire continuamente query di lettura tramite i DAO avrebbe rallentato l'intera applicazione.

Per risolvere questo collo di bottiglia, abbiamo implementato **GridCluster**. Si tratta di una cache in memoria, gestita come Singleton, che viene popolata una sola volta

all'avvio dell'applicazione. Al suo interno, i dati non sono salvati in semplici liste, ma indicizzati strategicamente tramite mappe Hash per garantire tempi di accesso costanti ($O(1)$). Ad esempio, raggruppando le colonnine per ID del trasformatore, il sistema può recuperare l'intero ramo di rete istantaneamente:

```
private Map<Long, List<ChargingStation>> stationsByTransformer;

public void init(TransformerDao tDao, StationDao sDao) {
    this.transformers = tDao.findAll();
    this.allStations = sDao.findAll();

    for (PowerTransformer t : transformers) {
        stationsByTransformer.put(t.getId(), new ArrayList<>());
    }

    for (ChargingStation station : allStations) {
        Long tId = station.getTransformer().getId();
        if (stationsByTransformer.containsKey(tId)) {
            stationsByTransformer.get(tId).add(station);
        }
    }
}
```

Oltre alla gestione della rete elettrica, questa classe espone anche i metodi di ricerca geografica, utilizzando la distanza euclidea per restituire rapidamente all'utente le colonnine disponibili più vicine.

3.2.2 GridMonitor

Non avendo a disposizione dei sensori fisici che inviano dati telemetrici, avevamo bisogno di un componente che facesse scorrere il tempo in modo autonomo, senza bloccare l'interfaccia grafica.

Abbiamo quindi esteso la classe `Thread` di Java, impostandola come "Demone" nel costruttore (`this.setDaemon(true);`). In questo modo il ciclo vitale del monitor viene legato a quello dell'interfaccia JavaFX: se l'utente chiude la finestra dell'applicazione, il thread si arresta in modo pulito senza lasciare processi appesi in background.

Il ciclo di esecuzione principale si sveglia ogni 5 secondi e orchestra le operazioni dei vari servizi, aggregando i dati prima di applicarli:

```
private void onTick() {  
  
    expireShortTermHolds();  
  
    Map<PowerTransformer, Double> heatMap = processActiveSessions()  
        ;  
  
    processTransformers(heatMap);  
  
    checkRatingAlerts();  
}
```

In particolare, la fase di elaborazione (`processActiveSessions`) calcola il calore generato dalle singole strategie di ricarica in uso (Fast o Eco) e lo somma all'interno di una *Heat Map*, in modo da notificare i trasformatori una sola volta per ciclo, ottimizzando ulteriormente le prestazioni.

3.2.3 LoadBalancerService

Il `LoadBalancerService` è il componente per cui è necessaria l'esistenza del pattern Observer all'interno dell'architettura. Invece di interrogare attivamente la rete per trovare eventuali surriscaldamenti, questo servizio si registra semplicemente in ascolto sui trasformatori.

Quando riceve una notifica di `THERMAL_ALERT`, il servizio reagisce isolando immediatamente il ramo di rete compromesso. L'intervento si divide in due fasi distinte, con garanzie transazionali deliberatamente diverse. Nella **prima fase** viene aperta un'unica transazione che porta a `OVERLOADED` tutte le colonnine collegate al trasformatore, impedendo l'avvio di nuove ricariche: o passano tutte, o non ne passa nessuna. Questa transazione viene poi *chiusa prima* di passare alla seconda fase, e non per un problema di deadlock, ma per una ragione più semplice: la **seconda fase** delega la chiusura delle sessioni in corso al `SessionService`, che apre connessioni proprie: tenere aperta la connessione della prima fase mentre se ne aprono altre sulle stesse righe significherebbe contendere inutilmente i lock che quella transazione ha già acquisito. Chiuderla per tempo li rilascia.

Listing 3.8: Gestione dell’alert termico

```
private void handleThermalAlert(PowerTransformer transformer) {

    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        for (ChargingStation station : stations) {
            station.setOverloaded();
            stationDao.update(station);
        }
        connection.commit();
    } finally {
        closeQuietly(connection); // Previene blocchi a cascata
    }

    for (ChargingSession session : sessionService.getActiveSessions(
        )) {
        if (belongsToTransformer(session.getStation(), stations)) {
            sessionService.forceClose(session);
        }
    }
}
```

Questa seconda fase non è atomica nel suo complesso: ogni chiusura forzata (`forceClose`) apre e chiude una propria transazione indipendente, invece di essere racchiusa in un’unica transazione che copra tutte le sessioni del ramo. È una scelta voluta, non un limite del sistema: se il fallimento di un singolo rimborso facesse fallire anche gli altri già conclusi con successo, si penalizzerebbero driver che non hanno responsabilità in quel fallimento.

Ogni chiusura forzata produce due effetti economici distinti, salvati nella stessa transazione, il rimborso al Driver e l’accredito allo Station Operator.

Listing 3.9: Rimborso e accredito in forceClose()

```
double kwhLastTick = station.getPowerKw() * (5.0 / 3600.0);
BigDecimal refundAmount = BigDecimal.valueOf(
    strategy.calculateCost(kwhLastTick, station, session.getDriver
        ()));

if (wasActive && station.getOperator() != null) {
    double kwhDelivered = session.getKwhDelivered();
    BigDecimal operatorShare = station.getTariffOperator()
        .multiply(BigDecimal.valueOf(kwhDelivered))
        .setScale(2, RoundingMode.HALF_UP);
    operator.addEarnings(operatorShare);
    userDao.update(operator);
}
```

Il Driver viene rimborsato solo del costo dell'ultimo tick da 5 secondi, cioè l'energia pagata ma non ancora erogata al momento dello stacco. Lo Station Operator, allo stesso tempo, viene comunque accreditato della propria quota sull'intera energia realmente erogata fino a quel punto. L'interruzione non annulla quindi l'intera operazione: divide il conto tra le due parti in base a cosa è realmente accaduto prima dello stop.

Rientro in servizio delle colonnine

L'emergenza però non è uno stato definitivo, e il `LoadBalancerService` gestisce anche la fase di rientro. Il trasformatore non ha una soglia sola, ma **due**: una di *allarme* a 90 °C, oltre la quale scatta il `THERMAL_ALERT` appena descritto, e una di *rientro* più bassa, a 70 °C. Quando la temperatura, non essendoci più ricariche attive su quel ramo, scende nuovamente sotto i 70 °C, il trasformatore emette un secondo avviso, `COOLING_COMPLETE`, che lo stesso observer intercetta per riportare ad `ACTIVE` tutte e sole le colonnine rimaste in `OVERLOADED`, di nuovo in un'unica transazione.

Listing 3.10: Rientro in servizio dopo il raffreddamento

```
private void handleCoolingComplete(PowerTransformer transformer) {
    List<ChargingStation> stations =
        GridCluster.getInstance().getStationsForTransformer(
            transformer.getId());
    ...
    for (ChargingStation station : stations) {
        if (station.getStatus() == StationStatus.OVERLOADED) {
            station.setActive();
            stationDao.update(station);
        }
    }
    connection.commit();
    ...
}
```

La distanza tra le due soglie è fondamentale perchè evita che il sistema oscilli. Infatti se ci fosse una soglia sola, un trasformatore fermo esattamente sul valore critico riattiverebbe le colonnine, che ricomincerebbero subito a scaldare, facendolo risalire sopra la soglia e provocando un nuovo blocco. Si avrebbe sostanzialmente un ciclo di accensioni e spegnimenti a raffica. Imponendo che per rientrare si debba scendere *ben sotto* il punto in cui si è usciti, si ottiene un margine di raffreddamento reale prima di riaprire il ramo. Il filtro `if (station.getStatus() == OVERLOADED)` garantisce inoltre che il rientro riguardi solo le colonnine spente dall'emergenza, senza risvegliare quelle che si trovano in altri stati per ragioni indipendenti (per esempio sospese dall'Energy Manager per qualità scadente).

3.2.4 SessionService

Mentre il `LoadBalancerService` gestisce le emergenze, il `SessionService` governa l'operatività economica ed energetica: è il servizio più sollecitato del sistema, invocato ad ogni tick del `GridMonitor` per ciascuna sessione in corso.

Addebito progressivo del wallet durante la ricarica

il portafoglio del Driver non viene addebitato al termine della ricarica, ma progressivamente, cinque secondi alla volta, dal `GridMonitor`. Ogni tick calcola l'energia erogata nell'intervallo (la potenza della colonnina moltiplicata per la frazione d'ora corrispondente), ne chiede il costo alla `ChargingStrategy` in uso, e scala subito quella cifra dal saldo:

Listing 3.11: Avanzamento di una ricarica: addebito e persistenza in un'unica transazione

```

public void addTick(ChargingSession session, ChargingStrategy
strategy) {
    double oreTick = 5.0 / 3600.0;
    double kwhThisTick = session.getStation().getPowerKw() *
        oreTick;
    BigDecimal costThisTick = BigDecimal.valueOf(
        strategy.calculateCost(kwhThisTick, session.getStation(),
            session.getDriver()));

    session.addTick(kwhThisTick, costThisTick);    // stato della
        sessione (in memoria)
    Driver driver = session.getDriver();
    driver.charge(costThisTick);                    // saldo del
        driver (in memoria)

    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);           // <-- Unit of
            Work: una connessione, due DAO

        UserDao userDao = daoFactory.createUserDao(connection);
        SessionDao sessionDao = daoFactory.createSessionDao(
            connection);

        userDao.update(driver);
        sessionDao.update(session);

        connection.commit();                        // o entrambe le
            scritture, o nessuna
    } catch (Exception e) {
        rollbackQuietly(connection);
        throw new RuntimeException("Errore durante il salvataggio
            del tick di ricarica", e);
    } finally {
        closeQuietly(connection);
    }

    if (driver.getWalletBalance().compareTo(BigDecimal.ZERO) <= 0)
    {
        forceClose(session);                        // fondi esauriti
    } else if (session.getStatus() != SessionStatus.ACTIVE) {
        closeSession(session);                      // batteria al
            100%
    }
}

```

3.2.5 StationService

Sebbene gran parte di questo servizio si occupi di gestire il salvataggio e la validazione delle nuove colonnine (delegando i controlli di sicurezza ai Factory Method del dominio), è presente una complessità notevole nella gestione delle prenotazioni (*Short-Term Hold*).

Infatti il problema in fase di prenotazione è la concorrenza: due Driver potrebbero tentare di riservare la stessa colonnina nell'esatto momento in cui si libera. La soluzione ingenua, *leggo lo stato, controllo che sia **ACTIVE**, scrivo **RESERVED***, è una classica *race condition*: fra la lettura e la scrittura c'è una finestra in cui un secondo Driver può leggere lo stesso stato **ACTIVE** e prenotare a sua volta, e la colonnina finirebbe assegnata a entrambi.

Il servizio non fa quindi due operazioni separate, ma una sola: delega al DAO (`acquireAtomicHold`) un **aggiornamento condizionale**, in cui il controllo e la scrittura vivono nella stessa istruzione SQL e vengono quindi eseguiti atomicamente dal database.

Listing 3.12: La condizione di guardia vive dentro la UPDATE

```
UPDATE charging_stations
  SET status = 'RESERVED', reserved_by_id = ?,
      expiration_timestamp = ?
 WHERE id = ? AND status = 'ACTIVE'
```

È la clausola `AND status = 'ACTIVE'` a far funzionare il meccanismo: se un altro Driver è arrivato un istante prima, la riga non è più **ACTIVE**, la `UPDATE` non aggiorna alcuna riga e restituisce 0. Il DAO riporta quel 0 al servizio, che lo traduce in una `StationNotAvailableException` per il secondo utente.

In realtà a differenza di quanto si potrebbe intuire non si impedisce al secondo Driver di provarci, ma si fa in modo che il suo tentativo fallisca in modo pulito e riconoscibile.

Inoltre è bene specificare che l'applicazione è stata implementata per un utente alla volta, quindi questo scenario di corsa non può materialmente verificarsi nella demo. La prenotazione è l'unico punto del sistema in cui due utenti competerebbero per la stessa risorsa, ed è stata scritta per un eventuale sistema multiutente.

3.2.6 Sicurezza e Validazione

La responsabilità di sicurezza dell'applicazione è divisa in due aree distinte, gestite da servizi separati:

- **Autenticazione (AuthService):** Il sistema non salva mai le credenziali in chiaro. L'hashing delle password è delegato alla libreria esterna **BCrypt**. Inoltre, il flusso di registrazione varia dinamicamente in base al ruolo richiesto, imponendo l'assegnazione di coordinate GPS iniziali esclusivamente ai Driver.
- **Ammissione in rete (ValidationService):** Nessuna colonnina registrata da un operatore entra in rete automaticamente. Alla registrazione nasce in stato `PENDING_VERIFICATION` e vi resta finché un Energy Manager non la approva o la rifiuta.

Il secondo punto merita una precisazione, perché è facile leggere l'approvazione come un semplice "sì/no" amministrativo, mentre nel nostro modello è anche un *atto economico*:

approvare una colonnina significa fissarne la quota di piattaforma. Al Manager, al momento dell'approvazione, viene chiesta la tariffa in euro per kWh che la piattaforma tratterrà su ogni ricarica erogata da quella colonnina; da quel momento quel valore entra nel calcolo del prezzo di ogni sessione su quella colonnina. Il gestore della rete, cioè, non si limita ad autorizzare: fissa anche il prezzo che la piattaforma applica su ogni ricarica, mentre l'operatore resta libero di fissare il proprio.

La divisione delle responsabilità all'interno del flusso è la parte interessante. Il servizio *non* decide se la transizione di stato sia lecita: si limita a orchestrare la transazione e a chiedere al dominio di applicarla.

Listing 3.13: Approvazione: il Service orchestra, il dominio decide

```
public void approve(ChargingStation station, BigDecimal
platformTariff) {
    if (platformTariff == null || platformTariff.compareTo(
        BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException("La tariffa della
            piattaforma deve essere un valore valido e non negativo.
            ");
    }
    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        station.activate(platformTariff);    // LOGICA DI DOMINIO: e
        ' la stazione a validare la transizione
        stationDao.update(station);        // PERSISTENZA

        connection.commit();
    } catch (Exception e) {
        rollbackQuietly(connection);
        ...
    } finally {
        closeQuietly(connection);
    }
}
```

La regola *"si approva solo ciò che è in attesa"* non vive quindi nel `ValidationService`, ma dentro `ChargingStation.activate()`: è la colonnina stessa a sapere di poter diventare `ACTIVE` soltanto a partire da `PENDING_VERIFICATION`, e a sollevare un'eccezione se qualcuno provasse ad approvare una colonnina già attiva o già rifiutata. Il servizio si occupa solo di *quando* invocare la transizione e di garantirne la persistenza atomica: se il dominio rifiuta, il `rollback` annulla tutto e nulla arriva al database. La stessa struttura si ritrova nel percorso di rifiuto (`reject`), dove il dominio applica la transizione verso `REJECTED` dopo che il Manager ha fornito una motivazione obbligatoria.

3.2.7 Gestione Utente e Portafoglio

La gestione del profilo utente e del suo portafoglio è affidata al `DriverService` e al `WalletService`. Quest'ultimo, in particolare, implementa le funzionalità legate ai pagamenti e ai piani tariffari, sollevando la classe di dominio `Driver` dalla responsabilità di interfacciarsi con il database.

Una delle operazioni più delicate di questo modulo è il cambio di abbonamento, la cui logica di calcolo è già contenuta in `Driver.updatePlan()` (vedi paragrafo 3.1.1). Il `WalletService`, prima di delegare al dominio, effettua un'unica verifica aggiuntiva sul saldo:

Listing 3.14: Verifica del saldo prima dell'upgrade

```
if (delta.compareTo(BigDecimal.ZERO) > 0
    && driver.getWalletBalance().compareTo(delta) < 0) {
    throw new IllegalStateException(
        "Fondi insufficienti: servono EUR " + delta + " per l'
        upgrade a " + plan.name() + ".");
}
BigDecimal charged = driver.updatePlan(plan);
```

3.2.8 Gestione Feedback e Manutenzione

Il flusso di controllo della qualità del servizio è interamente gestito dal `RatingService`. Il ciclo di vita di un feedback inizia nel momento in cui il `Driver` conclude una ricarica e decide di valutare l'esperienza.

In questa prima fase, il sistema esegue un duplice controllo. Dapprima verifica a livello database che l'utente non abbia già recensito quella specifica sessione; superato questo filtro, delega la validazione del formato del voto (da 1 a 5 stelle) al dominio.

La parte interessante di questo servizio è la logica che si innesca subito dopo il salvataggio. Non essendoci un controllo umano costante sulle singole colonnine, il servizio ricalcola in tempo reale la media della stazione appena recensita. Se tale parametro scende sotto la soglia critica di 2.0 (con un campione minimo di 5 valutazioni), il sistema interviene autonomamente generando un `RatingAlert`.

Listing 3.15: Ricalcolo della media e alert di manutenzione

```
private void checkRatingAlert(Connection connection,
    ChargingStation station) {

    ratingDao.recalculateAverage(station.getId());
    ChargingStation updatedStation = stationDao.findById(station.
        getId());

    Double currentAvg = updatedStation.getAverageRating();
    Integer totalRatings = updatedStation.getTotalRatings();

    if (currentAvg != null && currentAvg < 2.0 && totalRatings !=
        null && totalRatings >= 5) {
        if (!ratingAlertDao.existsOpenAlertForStation(station.getId()
            ())) {
            RatingAlert alert = RatingAlert.create(updatedStation,
                currentAvg);
            ratingAlertDao.save(alert);
        }
    }
}
```

Una volta ricevuta la segnalazione sulla propria dashboard, sarà discrezione dell'Energy Manager decidere se ignorarla (qualora si trattasse di un falso allarme) o sospendere la stazione difettosa.

Nota: la mappa di Firenze e il Jitter

Per rendere i test più verosimili, abbiamo deciso di utilizzare come ambiente geografico l'area urbana di Firenze, mappando i veri quartieri (dal Centro Storico alla periferia) tramite l'enumerazione *Zone*.

Ci siamo però accorti subito di un problema: registrando più colonnine nello stesso quartiere, queste avrebbero assunto le stesse identiche coordinate di base, finendo per sovrapporsi perfettamente in caso di futura rappresentazione grafica. Per risolverlo, abbiamo introdotto una piccola funzione di *Jitter* all'interno dell'enum. Questo metodo calcola un microscopico scostamento casuale (corrispondente a circa 500 metri) e lo applica al centro esatto della zona, in modo tale da generare latitudini e longitudini leggermente diverse per ogni stazione, pur rimanendo all'interno del quartiere di riferimento.

Listing 3.16: Distribuzione geografica con jitter

```
public double[] resolveWithJitter() {

    double jLat = (Math.random() - 0.5) * 0.01;
    double jLng = (Math.random() - 0.5) * 0.01;
    return new double[]{ baseLat + jLat, baseLng + jLng };
}
```

3.3 Controllers

Ogni schermata dell'applicazione ha un Controller associato, il cui compito è sempre lo stesso: leggere cosa ha fatto l'utente, per esempio un click o un testo inserito, passare la richiesta al Service giusto del Business Layer, e mostrare il risultato a schermo. Per questo motivo la struttura interna dei Controller è quasi identica tra loro, e di seguito ne presentiamo l'elenco raggruppato per area applicativa, con una breve descrizione del ruolo di ciascuno.

Autenticazione

- **LoginController:** Raccoglie email e password e delega l'autenticazione ad **AuthService**, instradando l'utente verso la dashboard corretta in base al ruolo.
- **RegistrationController:** Raccoglie i dati per un nuovo account, incluso il ruolo scelto (Driver, Station Operator, Energy Manager).

Area Driver

- **DriverMenuController:** Contenitore della dashboard Driver: gestisce il menu laterale e sostituisce solo il contenuto centrale ad ogni navigazione.
- **StationExplorerController:** Mostra le colonnine compatibili nelle vicinanze e permette di prenotarne una o avviare direttamente una sessione.
- **LiveSessionController:** Mostra l'avanzamento in tempo reale di una ricarica in corso. Non calcola nulla e non è un observer del **GridMonitor**: rilegge lo stato della sessione dal database una volta al secondo, tramite una **Timeline JavaFX** indipendente dal tick del monitor (si veda il paragrafo 3.3).
- **ChargingHistoryController:** Elenca le sessioni passate del Driver, completate o interrotte per allarme termico.
- **FundWalletController:** Gestisce la ricarica del saldo del wallet.
- **AccountController:** Mostra i dati del profilo: piano di abbonamento (modificabile), email e veicolo (in sola lettura).

Area Station Operator

- **OperatorMenuController:** Contenitore della dashboard dell'operatore, da cui si accede alla gestione delle colonnine e ai rendiconti economici.
- **NewStationController:** Modulo di registrazione di una nuova colonnina, con i suoi parametri tecnici.
- **StationHubController:** Elenco dettagliato delle colonnine possedute dall'operatore, con stato operativo di ciascuna (online, in sovraccarico, ecc.) e accesso rapido alla registrazione di una nuova.

- **FinancialReportsController:** Riepilogo economico dell'operatore: guadagno totale, energia venduta e numero di sessioni erogate.

Area Energy Manager

- **EnergyManagerMenuController:** Contenitore della dashboard dell'Energy Manager, da cui si accede alle altre schermate di gestione della rete.
- **ApprovalQueueController:** Coda delle colonnine in attesa di verifica: l'Energy Manager può approvarle o rifiutarle.
- **RatingIncidentManagerController:** Elenco degli alert generati automaticamente quando una colonnina riceve valutazioni troppo basse, con le azioni di sospensione o chiusura dell'alert.
- **ReliabilityDashboardController:** Stato in tempo reale della rete elettrica: carico e temperatura di ogni trasformatore, con evidenza visiva di eventuali allarmi termici.

Come un Controller ottiene i suoi servizi: il Service Locator

C'è un vincolo tecnico da cui non si scappa: **i Controller non li istanziamo noi**. È JavaFX a costruirli, via reflection, mentre carica il file `.fxml` a cui sono associati. Questo esclude la tecnica che sarebbe la più pulita per fornire a un oggetto le sue dipendenze, cioè passargliele nel costruttore (*dependency injection*): il costruttore non lo chiamiamo noi.

La soluzione adottata è il pattern **Service Locator**. Quando un controller ha bisogno di un servizio non se lo fa passare, ma va a chiederlo a un punto centrale noto, il `NavigationManager`, che espone metodi come `getSessionService()`, `getStationService()` e `getCurrentUser()`. Il recupero avviene nel metodo `initialize()`, che JavaFX invoca subito dopo aver costruito il controller e iniettato i campi `@FXML`.

È bene notare la coerenza della scelta, perché è il punto in cui questo capitolo si collega direttamente al Capitolo 4: i *Controller* usano il Service Locator *perché non siamo noi a istanziarli*, ma i *Service* fra loro continuano a usare la classica dependency injection (ricevono la `DaoFactory` nel costruttore), perché i *Service* li creiamo noi, e passare le dipendenze dall'esterno è esattamente ciò che permette di sostituirle con dei mock e testarli in isolamento.

Un principio che ne discende, tenuto fermo in tutto il progetto: **un Controller non parla mai direttamente al database**. Passa sempre da un *Service*. Un pezzo di interfaccia che scavalcasse la logica di business per andare dritto a un DAO renderebbe impossibile far rispettare le regole (validazioni, calcoli, transizioni di stato) che nel DAO non esistono.

Il polling della Live Session, indipendente dal GridMonitor

La schermata di ricarica in tempo reale (`LiveSessionController`) è quella in cui il rapporto fra interfaccia e simulazione si vede meglio, ed è anche quella che si presta al fraintendimento più naturale: verrebbe da pensare che la schermata sia un *observer* del `GridMonitor`, notificata ad ogni tick. Non è così, e la distinzione è deliberata.

La schermata **non calcola nulla e non riceve alcuna notifica**: possiede un proprio ciclo di aggiornamento, una `Timeline` JavaFX che una volta al secondo rilegge dal database lo stato della sessione e aggiorna i numeri a video.

Listing 3.17: Il ciclo di lettura della schermata, indipendente dal tick del monitor

```
// Il GridMonitor fa girare i tick, noi osserviamo
pollTimeline = new Timeline(new KeyFrame(Duration.seconds(1), e ->
    refreshFromDb()));
pollTimeline.setCycleCount(Timeline.INDEFINITE);
pollTimeline.play();
```

Il `GridMonitor` scrive ogni 5 secondi, la schermata legge ogni secondo, e i due cicli non comunicano mai direttamente: l'unico punto di sincronizzazione è il database, che resta la sola fonte di verità. Il vantaggio è concreto: **se il Driver chiude la schermata, la ricarica prosegue lo stesso**, esattamente come una colonnina reale continua a caricare anche quando non si guarda l'app sul telefono. Per rientrare, il menu del Driver mostra una scorciatoia "Resume" verso la sessione ancora in corso. Se la schermata fosse stata un observer del motore di simulazione, chiuderla avrebbe significato staccare l'ascoltatore, e sarebbe stato molto più facile scrivere per sbaglio un'interfaccia che, chiudendosi, ferma la simulazione.

C'è infine un accorgimento tecnico che questa scelta ha reso necessario. Quando la ricarica termina, la schermata deve capire *perché*: conclusione normale (batteria carica o stop volontario) oppure interruzione d'autorità per surriscaldamento, caso in cui va mostrata una finestra di emergenza. Il problema è che JavaFX non consente di aprire una finestra modale mentre è in esecuzione un'animazione, e una `Timeline` è a tutti gli effetti un'animazione. La comparsa del messaggio è quindi stata rinviata al ciclo di eventi immediatamente successivo, impacchettandola in `Platform.runLater(...)`. È un punto che si presta a un equivoco che conviene prevenire: qui `runLater` serve *esclusivamente* a rimandare il dialogo fuori dal ciclo della `Timeline`, **non** a spostare l'aggiornamento da un thread di background al thread grafico, perché la `Timeline` gira già sul thread di JavaFX.

Il dato sempre aggiornato

Un dettaglio implementativo interessante si trova in `FinancialReportsController`: prima di leggere il guadagno totale dell'operatore, il controller non si fida del valore tenuto in sessione, ma lo rilegge direttamente dal database.

Listing 3.18: Aggiornamento del controller con dati freschi

```
User refreshed = NavigationManager.getAuthService().refreshUser(
    loggedOperator.getId());
if (refreshed instanceof StationOperator op) {
    this.loggedOperator = op;
}
BigDecimal revenue = loggedOperator.getTotalEarnings() != null
    ? loggedOperator.getTotalEarnings() : BigDecimal.ZERO;
```

L'oggetto `User` tenuto in sessione contiene una *fotografia* dei dati dell'utente al momento del login. Alcuni di quei dati cambiano però mentre l'utente naviga: il saldo del Driver scende ad ogni tick di ricarica, i guadagni dell'operatore salgono ad ogni sessione chiusa. Una schermata che mostrasse il valore in memoria mostrerebbe un numero vecchio. La regola, applicata a tutte le schermate che espongono valori mutevoli, è che l'identità e il

ruolo si leggono dalla sessione, mentre qualunque valore soggetto a variazione si rilegge dal Service competente ogni volta che la schermata viene aperta. Questa convenzione evita un'intera categoria di bug, quelli in cui l'utente vede numeri non aggiornati.

3.4 Persistenza: il livello DAO e il mapping oggetti/tabelle

Il livello DAO si occupa di tradurre le righe delle tabelle SQL in oggetti Java e viceversa. È bene chiarire subito la terminologia, perché il problema che questo livello risolve, il disallineamento fra il modello a oggetti e quello relazionale, è lo stesso che affrontano gli ORM (*Object-Relational Mapper*) come Hibernate: la differenza è che qui **non è stato usato alcun ORM**. Il mapping è scritto a mano, riga per riga, con query SQL esplicite e `PreparedStatement`. È una scelta didattica deliberata: costringe a vedere esattamente cosa accade fra un oggetto Java e una riga di tabella, invece di delegarlo a una libreria che lo nasconde.

Il `GenericDao<T>` definisce le operazioni base che garantiscono la persistenza dei dati:

Listing 3.19: Interfaccia DAO generica

```
public interface GenericDao<T> {  
    void save(T entity);  
    T findById(Long id);  
    List<T> findAll();  
    void update(T entity);  
    void delete(Long id);  
}
```

Ogni entità del dominio ha poi una classe concreta (es. `PostgresUserDao`) che implementa queste operazioni scrivendo query SQL vere tramite `PreparedStatement`, ed esegue la trasformazione in entrambe le direzioni: dall'oggetto Java verso le colonne della tabella al momento del salvataggio, e dalle colonne del `ResultSet` di nuovo verso un oggetto Java al momento della lettura.

Tabella users

La tabella `users` contiene sia i Driver che gli Station Operator che gli Energy Manager, distinti da una colonna `role`. Il mapping deve quindi decidere, riga per riga, quale sottoclasse istanziare:

Listing 3.20: Mapping utente da ResultSet

```
private User extractUserFromResultSet(ResultSet rs) throws
SQLException {
    Long id = rs.getLong("id");
    String name = rs.getString("name");
    String dbEmail = rs.getString("email");
    String password = rs.getString("password");
    Role role = Role.valueOf(rs.getString("role"));

    if (role == Role.DRIVER) {
        Double lat = rs.getDouble("latitude"); if (rs.isNull())
            lat = null;
        Double lon = rs.getDouble("longitude"); if (rs.isNull())
            lon = null;
        Double cap = rs.getDouble("battery_capacity"); if (rs.
            wasNull()) cap = null;
        String connTypeStr = rs.getString("connector_type");
        var cType = connTypeStr != null ? ConnectorType.valueOf(
            connTypeStr) : null;
        String subPlanStr = rs.getString("subscription_plan");
        var sPlan = subPlanStr != null ? SubscriptionPlan.valueOf(
            subPlanStr) : null;
        BigDecimal balance = rs.getBigDecimal("wallet_balance");
        if (balance == null) balance = BigDecimal.ZERO;
        return Driver.reconstitute(id, name, dbEmail, password, lat
            , lon, cType, sPlan, cap, balance);

    } else if (role == Role.STATION_OPERATOR) {
        BigDecimal earnings = rs.getBigDecimal("total_earnings");
        if (earnings == null) earnings = BigDecimal.ZERO;
        return StationOperator.reconstitute(id, name, dbEmail,
            password, earnings);

    } else if (role == Role.ENERGY_MANAGER) {
        return EnergyManager.reconstitute(id, name, dbEmail,
            password);
    }
    return null;
}
```

Colonne come `wallet_balance` o `total_earnings` esistono nella tabella ma hanno senso solo per un tipo di utente: per un Energy Manager, ad esempio, restano NULL. Il DAO legge la colonna `role` e usa quel valore per capire quale oggetto Java ricostruire.

Metodo `reconstitute()`

Il mapping non chiama mai `new Driver(...)` direttamente, ma un metodo chiamato `reconstitute()`. La differenza è importante: quando un Driver viene creato per la prima volta (in fase di registrazione) deve passare da un Factory Method che valida i dati in ingresso, perché in quel momento i dati arrivano da un utente e potrebbero essere

sbagliati. Quando invece un Driver viene riletto dal database, i dati sono già stati validati in passato (e protetti da vincoli SQL come i `CHECK` dello schema) — rivalidarli ad ogni lettura sarebbe inutile. `reconstitute()` si limita quindi a ricostruire l'oggetto così com'è, senza ripetere controlli già superati.

Ricostruzione parziale (stub)

A volte il DAO non ha bisogno di ricostruire un oggetto per intero, ma solo di una sua parte. Nel recupero di un `RatingAlert`, ad esempio, non serve caricare l'intera colonnina collegata (tariffe, stato, coordinate...): basta sapere il suo nome, da mostrare nella schermata dell'Energy Manager.

Listing 3.21: Ricostruzione parziale di `ChargingStation`

```
ChargingStation station = ChargingStation.reconstitute(
    stId, null, null, stName, null, null, null, null, null,
    null,
    null, null, null, null, null, null, null
);
return RatingAlert.reconstitute(alertId, station, avgAtCreation,
    status, managerNote, createdAt);
```

Questa `ChargingStation` è un oggetto reale (uno "stub"), di cui vengono valorizzati solo i campi necessari, mentre il resto resta `null`. In questo modo si evita una query aggiuntiva per recuperare dati che non verrebbero comunque mostrati; se però in futuro venisse ripassato a un metodo che si aspetta una colonnina completa (ad esempio per un aggiornamento), andrebbe prima ricaricato per intero.

I vincoli dello schema

Oltre alla validazione fatta dal codice Java, anche lo schema del database applica i propri controlli, indipendentemente da cosa succede nel Business Layer:

Listing 3.22: Vincoli SQL di integrità

```
wallet_balance DECIMAL(10, 2) CHECK (wallet_balance IS NULL OR
    wallet_balance >= 0),
...
stars INTEGER NOT NULL CHECK (stars >= 1 AND stars <= 5),
...
CONSTRAINT uq_driver_session UNIQUE (driver_id, session_id)
```

Se per un errore di programmazione un service costruisse un saldo negativo o un voto fuori dal range 1-5, sarebbe comunque il database a rifiutare l'inserimento: è un secondo controllo, indipendente da eventuali bug nel codice Java.

3.5 Idiomi e Design Pattern

Per risolvere alcuni problemi specifici e per rendere il codice più aperto a future estensioni, sono stati usati alcuni design pattern.

3.5.1 Observer Pattern

Quando un trasformatore (`PowerTransformer`) si scalda troppo si limita a lanciare un avviso, mentre chi lo riceve e decide cosa fare è `LoadBalancerService`.

Listing 3.23: Interfacce Subject e Observer

```
public interface Subject {
    void attach(Observer observer);
    void detach(Observer observer);
    void notifyObservers(TransformerEvent event);
}

public interface Observer {
    void update(Subject source, TransformerEvent event);
}

public enum TransformerEvent {
    THERMAL_ALERT,
    COOLING_COMPLETE
}
```

Il vantaggio è che i due componenti restano indipendenti: per aggiungere un secondo componente che si limiti a registrare gli allarmi, basterebbe iscriverlo come nuovo observer, senza modificare il codice del trasformatore.

3.5.2 Strategy Pattern

Un Driver può scegliere tra due modalità di ricarica, Fast o Eco, e ciascuna ha un proprio modo di calcolare il costo e il calore prodotto. Invece di usare un `if/else`, che andrebbe modificato ad ogni modalità aggiunta in futuro, si è definita un'unica interfaccia comune che ogni modalità implementa diversamente. In questo modo `SessionService` si interfaccia senza sapere quale modalità di ricarica ci sia dietro.

Listing 3.24: Interfaccia delle strategie di ricarica

```
public interface ChargingStrategy {
    double calculateCost(double kwh, ChargingStation station,
        Driver driver);
    double getHeatIncrement();
    ChargingType getType();
}
```

3.5.3 Factory Method

Per evitare che si creino oggetti senza rispettare le regole del dominio, la creazione di un oggetto passa sempre da un metodo che prima controlla che tutto sia in regola, e solo dopo costruisce l'oggetto:

Listing 3.25: Factory Method per la creazione del rating

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session,
        Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un
            valore compreso tra 1 e 5 stelle.");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Lo stesso principio si ritrova quando si apre una sessione di ricarica, quando si registra una nuova colonnina o quando si crea un alert di qualità: in tutti questi casi il controllo e la creazione avvengono insieme, così è impossibile ottenere un oggetto "a metà", che esiste ma è in uno stato che non dovrebbe essere permesso.

Listing 3.26: altro esempio di Factory method, qui è usato per creare alert di rating. A differenza degli esempi precedenti, qui il costruttore non deve validare nulla, i dati arrivano già controllati da chi lo invoca, ma resta comunque privato per impedire di costruire un alert in uno stato diverso da PENDING.

```
private RatingAlert(ChargingStation station, Double
    avgAtCreation) {
    this.station = station;
    this.avgAtCreation = avgAtCreation;
    this.status = RatingAlertStatus.PENDING;
    this.managerNote = null;
    this.createdAt = LocalDateTime.now();
}

public static RatingAlert create(ChargingStation station, Double
    avgAtCreation) {
    return new RatingAlert(station, avgAtCreation);
}
```

3.5.4 Singleton

Il sistema tiene traccia di quali colonnine sono collegate a quale trasformatore. Questa informazione viene consultata ogni 5 secondi dal componente che simula il passare del tempo `GridMonitor` e ogni volta che un trasformatore si surriscalda dal componente che deve bloccare le colonnine.

Se questa informazione fosse duplicata in due copie diverse, si rischierebbe un disallineamento tra i due componenti. La soluzione è garantire che ne esista una sola copia in tutto il programma, caricata una volta sola all'avvio e poi consultata da tutti:

Listing 3.27: Singleton GridCluster per la cache della rete

```
public class GridCluster {

    private static GridCluster instance;

    private List<PowerTransformer> transformers;
    private Map<Long, List<ChargingStation>> stationsByTransformer;
    private List<ChargingStation> allStations;

    private GridCluster() {
        this.transformers = new ArrayList<>();
        this.stationsByTransformer = new HashMap<>();
        this.allStations = new ArrayList<>();
    }

    public static synchronized GridCluster getInstance() {
        if (instance == null) {
            instance = new GridCluster();
        }
        return instance;
    }

    public void init(TransformerDao tDao, StationDao sDao) {
        this.transformers = tDao.findAll();
        this.allStations = sDao.findAll();
        for (PowerTransformer t : transformers) {
            stationsByTransformer.put(t.getId(), new ArrayList<>());
        }
        for (ChargingStation station : allStations) {
            Long tId = station.getTransformer().getId();
            if (stationsByTransformer.containsKey(tId)) {
                stationsByTransformer.get(tId).add(station);
            }
        }
    }
}
```

La parola `synchronized` su `getInstance()` serve perché più componenti girano in parallelo su thread diversi: senza questa protezione, due richieste arrivate nello stesso istante potrebbero entrambe trovare `instance` ancora nulla e crearne due copie per errore. I dati non vengono letti dal database ad ogni richiesta: `init()` viene chiamato una sola volta all'avvio dell'applicazione, e da quel momento in poi `getStationsForTransformer()` risponde all'istante leggendo dalla mappa già in memoria, senza interrogare più il database. La cache non è però immutabile: quando un operatore registra una nuova colonnina, lo `StationService` la aggiunge esplicitamente al cluster (`registerNewStation`), così che la mappa in memoria resti allineata al database senza doverla ricostruire da capo.

3.5.5 DAO (Data Access Object)

Il pattern che struttura l'intero livello di persistenza è stato descritto nel paragrafo 3.4, ma merita di comparire anche qui, perché è a tutti gli effetti un design pattern e non un semplice dettaglio implementativo. Il problema che risolve è il disallineamento fra due mondi che parlano lingue diverse: da una parte oggetti Java con i loro riferimenti (una `ChargingSession` che *contiene* un `Driver` e una `ChargingStation`), dall'altra righe fatte di colonne e numeri. Il DAO è il traduttore, ed è l'*unico* punto del sistema che sa com'è fatto il database.

La separazione fra *interfaccia* (`StationDao`) e *implementazione concreta* (`PostgresStationDao`) non è pignoleria formale: è ciò che ha reso quasi gratuita la migrazione da H2 a PostgreSQL. Il resto del sistema conosce solo l'interfaccia e ignora quale implementazione lavori sotto; per questo nessun Service e nessun Controller si è accorto del cambio di database. La costruzione delle implementazioni concrete è a sua volta isolata in una `DaoFactory`, il cui unico compito è fabbricarle.

3.5.6 Unit of Work

Strettamente legato al DAO, e già visto all'opera nel paragrafo 3.2.4, c'è il modo in cui il progetto tratta le operazioni che toccano più tabelle insieme. Un tick di ricarica scrive sul saldo del Driver e sullo stato della sessione; la chiusura di una sessione scrive su ricevute, colonnina, sessione e guadagni dell'operatore. Sono scritture correlate: eseguirne solo una parte lascerebbe il sistema in uno stato incoerente (denaro scalato senza energia registrata, o viceversa).

Il pattern applicato è lo **Unit of Work**: il Service apre *una sola* connessione, disabilita l'auto-commit, e da quella connessione fabbrica tutti i DAO che gli servono. Un punto su cui è bene essere espliciti, perché è la domanda naturale: i metodi dei DAO **non ricevono la connessione come parametro**; la connessione viene loro iniettata al momento della creazione, tramite la factory (`daoFactory.createXxxDao(conn)`). Condividendo la connessione, più DAO condividono automaticamente la stessa transazione, e alla fine un unico `commit` fissa tutto oppure un `rollback` annulla ogni modifica parziale.

3.5.7 Service Locator

L'ultimo pattern è quello che governa l'accesso ai servizi da parte dei Controller, ed è stato discusso nel paragrafo 3.3. Lo si richiama qui perché la sua adozione non è un ripiego, ma una risposta precisa a un vincolo del framework: essendo JavaFX a istanziare i Controller via reflection durante il caricamento degli `.fxml`, la dependency injection via costruttore non è praticabile. Il `NavigationManager` funge quindi da punto centrale noto presso cui i Controller reperiscono i servizi di cui hanno bisogno.

Il contrasto con il resto del sistema è la parte da tenere a mente: **i Controller usano il Service Locator perché non siamo noi a costruirli; i Service usano la Dependency Injection perché siamo noi a costruirli**, e ricevere le dipendenze dall'esterno è precisamente ciò che li rende sostituibili con dei mock nei test (Capitolo 4).

Capitolo 4

Testing

4.1 Strategia di Verifica

I test sono eseguiti su due livelli:

- **Livello 1 – Business Logic:** Service, componenti core (`GridCluster`, `GridMonitor`) e Strategy, testati isolando ogni dipendenza esterna con Mockito.
- **Livello 2 — Persistence Layer:** i DAO Postgres, testati contro un database H2 reale in modalità `MODE=PostgreSQL`, per verificare che le query SQL scritte a mano funzionino davvero e non solo che le interazioni tra oggetti siano quelle attese.

4.1.1 Tecnologie e Strumenti

- **JUnit 5:** framework di esecuzione, con le annotazioni `@BeforeEach/@AfterEach` per il setup e cleanup di ogni test e `@BeforeAll/@AfterAll` per le risorse costose (come la connessione H2) condivise da un'intera classe.
- **Mockito:** usato non solo per mockare interfacce (`StationDao`, `SessionService`, ...) ma anche per mockare **metodi statici** tramite `Mockito.mockStatic()` — una necessità specifica di questo progetto, perché sia `DatabaseManager.getConnection()` sia `GridCluster.getInstance()` sono chiamate statiche che altrimenti aprirebbero una connessione reale o toccherebbero lo stato globale del Singleton durante un test unitario.
- **H2 in modalità `MODE=PostgreSQL`:** usato esclusivamente nei test dei DAO, con lo schema reale caricato all'avvio tramite `INIT=RUNSCRIPT FROM './src/main/resources/scheme.` — lo stesso file `scheme.sql` usato in produzione, quindi i test verificano le query contro la struttura di tabelle reale, non una sua approssimazione.

4.2 Livello 1: Business Logic

4.2.1 Logica pura, senza mock — `ChargingStrategiesTest`

Le strategie di ricarica (`FastChargingStrategy`, `EcoChargingStrategy`) hanno stesso input, stesso output e nessuna dipendenza esterna. Per questo motivo il loro test non usa

Mockito: costruisce oggetti di dominio reali con `reconstitute()` e confronta il risultato con un secondo calcolo indipendente, scritto direttamente nel test:

```
private double basePrice(double kwh, SubscriptionPlan plan) {
    double discount = (plan != null) ? plan.getDiscount() : 0.0;
    BigDecimal platAfter = TARIFF_PLAT.multiply(BigDecimal.ONE.
        subtract(BigDecimal.valueOf(discount)));
    BigDecimal rate = TARIFF_OP.add(platAfter);
    return rate.multiply(BigDecimal.valueOf(kwh)).setScale(2,
        RoundingMode.HALF_UP).doubleValue();
}

@Test
void testEcoCalculateCostWithSubscriptionDiscount() {
    double kwh = 100.0;
    Driver premiumDriver = testDriver(3L, "Anna", "anna@test.com",
        SubscriptionPlan.PREMIUM);
    double expectedPremium = round2(basePrice(kwh, SubscriptionPlan.
        PREMIUM) * 0.90);
    assertEquals(expectedPremium, ecoStrategy.calculateCost(kwh,
        mockStation, premiumDriver), 0.001,
        "PREMIUM: sconto piano sulla sola quota piattaforma,
        poi Eco 10%");
}
```

Riscrivere la formula nel test invece di limitarsi a un valore atteso hardcoded (es. `assertEquals(33.30, ...)`) ha un vantaggio: se le tariffe di test cambiano, il valore atteso si ricalcola automaticamente insieme a esse, e il test continua a verificare la *relazione* tra i numeri (lo sconto si applica solo alla quota piattaforma) invece di un singolo numero fisso che potrebbe diventare scorrelato dalla logica reale.

Va però riconosciuto il rovescio della medaglia, perché è un'obiezione legittima: duplicando la formula di produzione dentro l'oracolo del test, un errore *sistematico* nella formula verrebbe copiato in entrambe le copie e il test passerebbe lo stesso. Un oracolo scritto ricalcolando la stessa logica non può, da solo, dimostrarne la correttezza. La scelta resta comunque difendibile, a patto di essere chiari su cosa questi test verificano davvero: non il valore assoluto del prezzo, ma l'**invariante di dominio** che quel valore deve rispettare, cioè che lo sconto dell'abbonamento morda *solo* sulla quota della piattaforma e mai su quella dell'operatore. Non a caso l'oracolo non ricopia `priceFor()` riga per riga: ricostruisce il prezzo a partire dalle due componenti separate, e sarebbe quindi il primo a cadere se qualcuno, in produzione, applicasse per errore lo sconto anche alla quota dell'operatore.

Questa regola merita una precisazione, perché nel codice convivono due sconti diversi. Lo sconto *dell'abbonamento* si applica alla sola quota piattaforma; lo sconto *della modalità Eco* (un ulteriore 10%) si applica invece al prezzo base complessivo. Non è un'incoerenza: in entrambi i casi lo sconto è a carico della piattaforma, perché allo Station Operator viene comunque accreditata la sua quota piena sull'energia erogata (paragrafo 3.2.4). È la piattaforma a finanziare le promozioni, non chi possiede la colonnina.

4.2.2 Singleton e reset dello stato : GridClusterTest

`GridCluster` è un Singleton (Capitolo 3.5 – Idiomi e Design Pattern): la sua istanza sopravvive tra un test e l'altro se non viene esplicitamente distrutta, col rischio che i dati caricati da un test "contaminino" il test successivo. La soluzione adottata è azzerare il campo statico `instance` via reflection prima di ogni test:

```
@BeforeEach
void setUp() throws Exception {
    Field instanceField = GridCluster.class.getDeclaredField("
        instance");
    instanceField.setAccessible(true);
    instanceField.set(null, null);

    gridCluster = GridCluster.getInstance();
    transformerDaoMock = Mockito.mock(TransformerDao.class);
    stationDaoMock = Mockito.mock(StationDao.class);
}
```

Con l'istanza garantita pulita, il test su `getNearestAvailable()` verifica sia la logica di filtro (solo le colonnine `ACTIVE`) sia quella di ordinamento (per distanza crescente), usando dati costruiti ad hoc invece di un vero calcolo GPS:

```
List<ChargingStation> nearest = gridCluster.getNearestAvailable
    (0.0, 0.0);

assertEquals(2, nearest.size(), "Deve escludere le stazioni non
    ACTIVE");
assertEquals("Vicino", nearest.get(0).getName(), "La prima deve
    essere quella geometricamente piu vicina");
assertEquals("Lontano", nearest.get(1).getName(), "La seconda deve
    essere quella piu lontana");
```

4.2.3 Service e gestione transazionale : DriverServiceTest, LoadBalancerServ

I Service che scrivono sul database aprono e chiudono esplicitamente una transazione JDBC (`setAutoCommit(false)`, poi `commit()` o `rollback()` in caso di errore, sempre seguito da `close()`). Testare questo comportamento richiede di intercettare la connessione prima che il Service la apra davvero — ed è qui che entra in gioco il mock statico:

Listing 4.1: Mock statico di DatabaseManager per intercettare la connessione JDBC.

```
@BeforeEach
void setUp() throws SQLException {
    connectionMock = mock(Connection.class);
    mockedDbManager = mockStatic(DatabaseManager.class);
    mockedDbManager.when(DatabaseManager::getConnection).thenReturn(
        (connectionMock));

    daoFactoryMock = mock(DaoFactory.class);
    userDaoMock = mock(UserDao.class);
    when(daoFactoryMock.createUserDao(connectionMock)).thenReturn(
        userDaoMock);

    driverService = new DriverService(daoFactoryMock);
}

@AfterEach
void tearDown() {
    if (mockedDbManager != null) {
        mockedDbManager.close(); // fondamentale: rilascia il mock
                                // statico globale
    }
}
```

Con la connessione sotto controllo, si possono verificare separatamente il percorso di successo e quello di fallimento — due test speculari che verificano esattamente le proprietà opposte sulla stessa transazione:

```
@Test
void testUpdateDriverLocation_Success() throws SQLException {
    Driver mockDriver = mock(Driver.class);
    driverService.updateDriverLocation(mockDriver, 43.7696,
        11.2558);

    verify(mockDriver, times(1)).updatePosition(43.7696, 11.2558);
    verify(userDaoMock, times(1)).update(mockDriver);
    verify(connectionMock, times(1)).commit();
    verify(connectionMock, times(1)).close();
}
```

```

@Test
void testUpdateDriverLocation_Failure() throws SQLException {
    doThrow(new RuntimeException("Database error")).when(
        userDaoMock).update(any(Driver.class));

    assertThrows(RuntimeException.class, () ->
        driverService.updateDriverLocation(mockDriver, 0.0, 0.0));

    verify(connectionMock, times(1)).rollback();
    verify(connectionMock, times(1)).close();
    verify(connectionMock, never()).commit();
}

```

Questa coppia di test è particolarmente significativa perché non si limita a verificare "il metodo non lancia errori": verifica che, in caso di eccezione, il sistema non lasci una transazione a metà — un dettaglio che un test puramente Black-Box (solo sul valore di ritorno) non potrebbe cogliere.

`LoadBalancerServiceTest` applica la stessa tecnica ma ne combina due insieme, mockando sia `DatabaseManager` sia il Singleton `GridCluster` nello stesso test, per isolare completamente la reazione a un evento `THERMAL_ALERT`:

Listing 4.2: Doppio mock statico: connessione JDBC e Singleton `GridCluster`.

```

mockedDbManagerStatic = mockStatic(DatabaseManager.class);
mockedDbManagerStatic.when(DatabaseManager::getConnection).
    thenReturn(connectionMock);

gridClusterMock = mock(GridCluster.class);
mockedGridClusterStatic = mockStatic(GridCluster.class);
mockedGridClusterStatic.when(GridCluster::getInstance).thenReturn(
    gridClusterMock);

```

Il test sul `THERMAL_ALERT` verifica in un colpo solo tre effetti attesi del pattern Observer descritto nel Capitolo 3.5: che la colonnina venga marcata come sovraccarica, che la modifica sia persistita, e che la chiusura delle sessioni attive sia *delegata* a `SessionService` invece di essere gestita internamente:

```

loadBalancerService.update(transformerMock, TransformerEvent.
    THERMAL_ALERT);

verify(stationMock, times(1)).setOverloaded();
verify(stationDaoMock, times(1)).update(stationMock);
verify(connectionMock, times(1)).commit();
verify(sessionServiceMock, times(1)).forceClose(sessionMock);

```

L'ultima riga è la più importante: verifica che `LoadBalancerService` non sappia *come* si chiude una sessione (rimborso, aggiornamento wallet), sappia solo *che* va chiusa — la stessa separazione di responsabilità già discussa a proposito di `LoadBalancerService` nel capitolo sul Business Layer.

4.2.4 Il cuore economico: SessionServiceTest

Il `SessionService` è il servizio più esposto del sistema, perché viene invocato ad ogni tick per ogni ricarica in corso ed è l'unico che muove denaro. È quindi anche il più testato (19 test). Ne riportiamo due, scelti perché verificano due proprietà che una semplice ispezione del codice non garantirebbe.

Il primo riguarda **dove** vengono fatti i controlli di business. Un Driver senza fondi non deve poter aprire una sessione: la regola vive nel dominio (`ChargingSession.open()`), e il test verifica non solo che l'eccezione venga lanciata, ma che ciò accada *prima* di toccare il database:

Listing 4.3: La validazione di dominio precede l'accesso al database

```
InsufficientBalanceException ex = assertThrows(  
    InsufficientBalanceException.class, () ->  
        sessionService.openSession(poorDriver, station, strategy,  
            20.0)  
);  
assertTrue(ex.getMessage().contains("insufficiente"));  
  
// Il fallimento avviene PRIMA di toccare il DB  
verify(daoFactoryMock, never()).createSessionDao(any());
```

L'ultima riga è la parte interessante: `verify(..., never())` sulla factory dei DAO non verifica un risultato, ma un *non-evento*. Fissa per contratto che una sessione senza copertura economica non arrivi nemmeno ad aprire una connessione: un test black-box sul solo valore di ritorno non potrebbe distinguere questo caso da uno in cui il database viene interrogato e poi il lavoro annullato.

Il secondo test verifica l'invariante contabile più importante del sistema, cioè che **allo Station Operator venga accreditata la sua quota, e solo la sua**. La quota è definita come tariffa operatore $\times$ kWh erogati; sul valore effettivamente accreditato si usa un `ArgumentCaptor`:

Listing 4.4: Cattura dell'importo accreditato all'operatore alla chiusura

```
ChargingSession session = ChargingSession.open(driver, station,  
    ChargingType.FAST, 20.0);  
session.addTick(10.0, new BigDecimal("5.00")); // kWh erogati =  
    10.0  
  
sessionService.closeSession(session);  
  
ArgumentCaptor<BigDecimal> captor = ArgumentCaptor.forClass(  
    BigDecimal.class);  
verify(fullOperator, times(1)).addEarnings(captor.capture());  
assertEquals(0, captor.getValue().compareTo(new BigDecimal("3.00"))  
    ,  
    "La quota deve essere 0.30 x 10 = 3.00");
```

Due dettagli meritano attenzione. Il primo è il confronto: si usa `compareTo(...)`

`== 0` e non `assertEquals` diretto sui `BigDecimal`, perché `BigDecimal.equals()` confronta anche la *scala*, e considererebbe `3.00` diverso da `3.0`. Su un test che verifica denaro, un confronto ingenuo produrrebbe falsi allarmi: si confrontano i *valori*, non la loro rappresentazione. Il secondo è la scelta di `times(1)`: insieme ai test gemelli `testCloseSession_NoOperator_NoCredit` e `testForceClose_CreditsOperator`, questo verifica la guardia `wasActive` discussa nel paragrafo 3.2.4, cioè che l'operatore non possa essere pagato due volte per la stessa energia.

4.2.5 La concorrenza sulle prenotazioni: `StationServiceTest`

L'altro punto delicato è l'acquisizione di una prenotazione, dove il servizio si affida a un update condizionale del database anziché a una coppia lettura/scrittura. Il test simula lo scenario di corsa mockando la risposta del DAO: `acquireAtomicHold` restituisce `false`, cioè "la `UPDATE` non ha aggiornato alcuna riga, qualcun altro è arrivato prima".

Listing 4.5: Il perdente della corsa non prenota, e non lascia una transazione a metà

```
// Simuliamo che qualcun altro abbia già preso la colonnina
when(stationDaoMock.acquireAtomicHold(100L, 1L)).thenReturn(false);

StationNotAvailableException exception = assertThrows(
    StationNotAvailableException.class, () ->
        stationService.hold(mockDriver, mockStation));

verify(mockStation, never()).setReserved(any()); // niente
mutazione in memoria
verify(connectionMock, never()).commit(); // niente commit
verify(connectionMock, times(1)).rollback(); // e la
transazione viene chiusa pulita
```

Le tre `verify` finali sono il vero contenuto del test, e verificano tre proprietà distinte: che l'oggetto in memoria non venga sporcato con una prenotazione che il database ha rifiutato (altrimenti memoria e DB divergerebbero), che nulla venga confermato, e che la transazione venga esplicitamente annullata invece di essere semplicemente abbandonata. Il test speculare (`testHold_Success`) verifica il percorso opposto sulla stessa transazione.

4.3 Livello 2: Persistence Layer (DAO)

I test dei DAO non usano mock: interrogano un database H2 reale, inizializzato ad ogni classe di test con lo stesso `schema.sql` usato in produzione.

Listing 4.6: Avvio di H2 con lo schema reale di produzione.

```
@BeforeAll
static void startDatabase() throws SQLException {
    String url = "jdbc:h2:mem:chargenet_test;MODE=PostgreSQL;
        DATABASE_TO_LOWER=TRUE;"
        + "DEFAULT_NULL_ORDERING=HIGH;INIT=RUNSCRIPT FROM
        './src/main/resources/scheme.sql'";
    connection = DriverManager.getConnection(url, "sa", "");
}
```

Poiché la connessione è condivisa da tutta la classe (@BeforeAll, non @BeforeEach, per non pagare il costo di riavviare H2 ad ogni singolo test), ogni test deve invece garantire di ripartire da una tabella vuota. Questo pone un problema: la tabella `charging_sessions` ha una foreign key verso `charging_stations`, quindi uno svuotamento diretto verrebbe rifiutato dal database finché non si disattivano temporaneamente i vincoli:

Listing 4.7: Pulizia delle tabelle rispettando temporaneamente i vincoli di integrità.

```
try (Statement stmt = connection.createStatement()) {
    stmt.execute("SET REFERENTIAL_INTEGRITY FALSE");

    stmt.execute("TRUNCATE TABLE charging_sessions RESTART IDENTITY
        ");
    stmt.execute("TRUNCATE TABLE charging_stations RESTART IDENTITY
        ");
    stmt.execute("TRUNCATE TABLE users RESTART IDENTITY");
    stmt.execute("TRUNCATE TABLE power_transformers RESTART
        IDENTITY");

    // ... reinserimento dei dati minimi necessari (operatore,
        driver, trasformatore, stazione) ...

    stmt.execute("SET REFERENTIAL_INTEGRITY TRUE");
}
```

Il vantaggio di questo approccio rispetto a un semplice `DELETE FROM: RESTART IDENTITY` azzerava anche i contatori auto-incrementali, così ogni test parte con ID prevedibili

Capitolo 5

Demo